LUMS

CENTER FOR SPEECH AND LANGUAGE TECHNOLOGIES

Motivating Decision Trees
Continuing from K-D Trees

Agha Ali Raza

CS535/EE514 – Machine Learning

Sources

Nearest Neighbor Methods, Victor Lavrenko, Assistant Professor at the University of Edinburgh, https://www.youtube.com/playlist?list=PLBv09BD7ez_48heon5Az-TsyoXVYOJtDZ

Machine Learning for Intelligent Systems, Kilian Weinberger, Cornell, Lecture 16, https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote16.html

Wiki K-Nearest Neighbors: https://en.wikipedia.org/wiki/K-nearest_neighbors_algorithm

Effects of Distance Measure Choice on KNN Classifier Performance - A Review, V. B. Surya Prasath et al., https://arxiv.org/pdf/1708.04321.pdf

A Comparative Analysis of Similarity Measures to find Coherent Documents, Mausumi Goswami et al. http://www.ijamtes.org/gallery/101.%20nov%20ijmte%20-%20as.pdf

A Comparison Study on Similarity and Dissimilarity Measures in Clustering Continuous Data, Ali Seyed Shirkhorshidi et al., https://journals.plos.org/plosone/article/file?id=10.1371/journal.pone.0144059&type=printable

Cover, Thomas, and, Hart, Peter. **Nearest neighbor pattern classification**. Information Theory, IEEE Transactions on, 1967, 13(1): 21-27

K-D Trees Algorithm

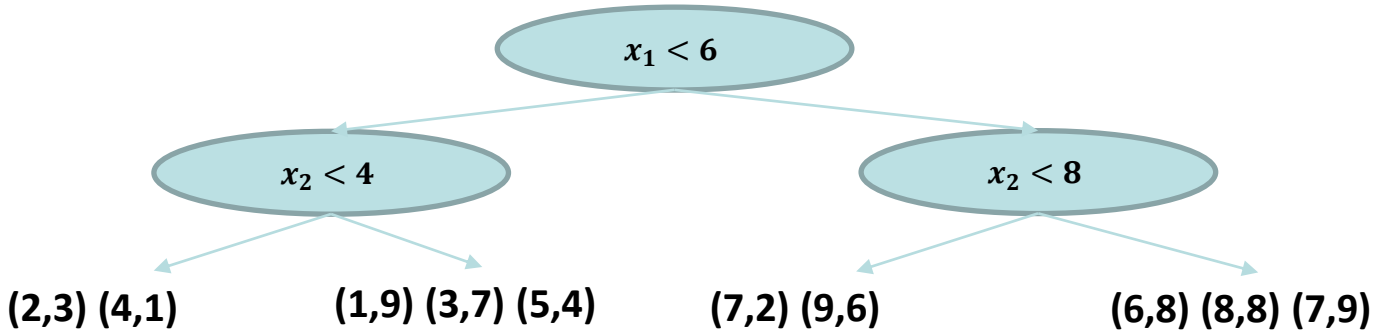
Pick a random dimension, find median, split data, repeat

$$(1,9), (2,3), (4,1), (3,7), (5,4), (6,8), (7,2), (8,8), (7,9), (9,6)$$

Make the K-D tree using training data:

1. Pick a new dimension randomly (or pick the dimension with the highest variance)
2. Find median
3. Split in equal halves around the median
4. Repeat

In the end we are left with all training data points in the leaves.

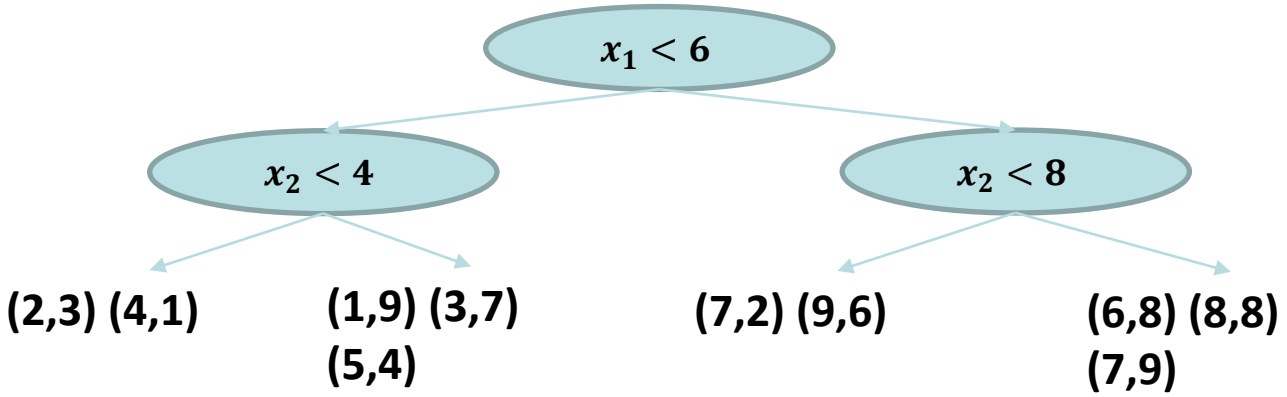


K-D Trees Algorithm

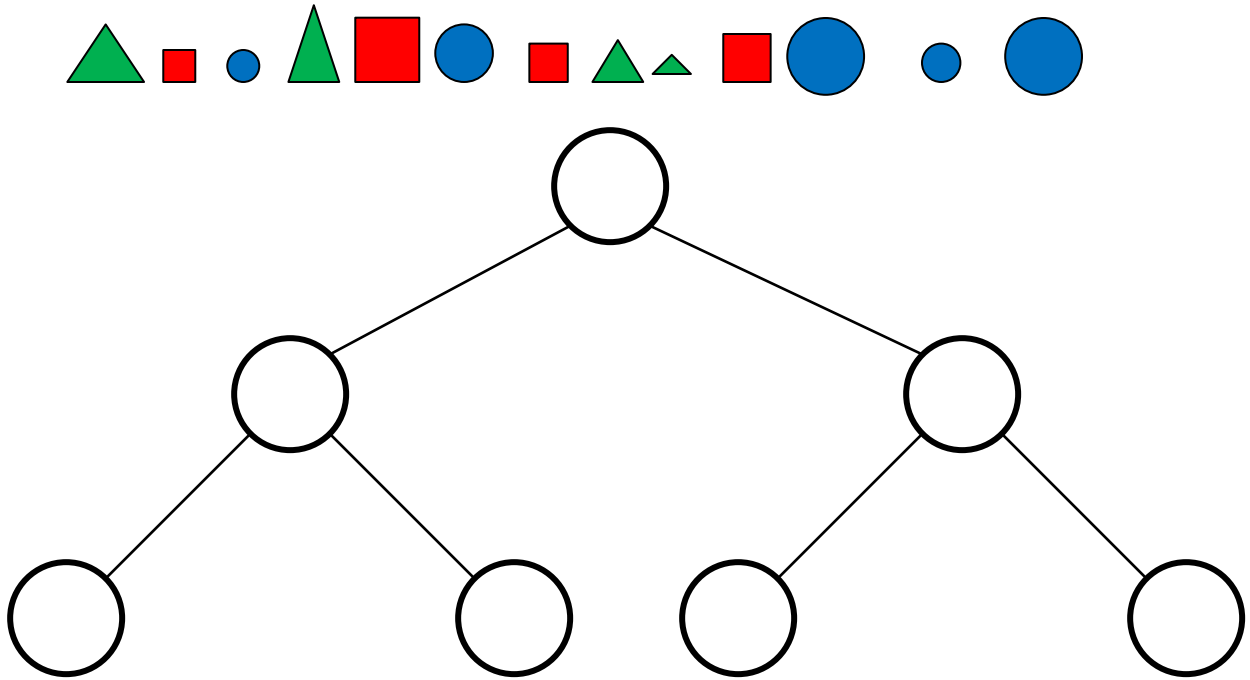
Finding the neighbors **for a test data point**:

1. Pass it down the tree, matching thresholds, until you reach a leaf
2. Find k nearest neighbors within that leaf

Note: This method may miss nearest neighbors.



Why do we still need nearest neighbors?



LUMS

CENTER FOR SPEECH AND LANGUAGE TECHNOLOGIES

Decision Trees

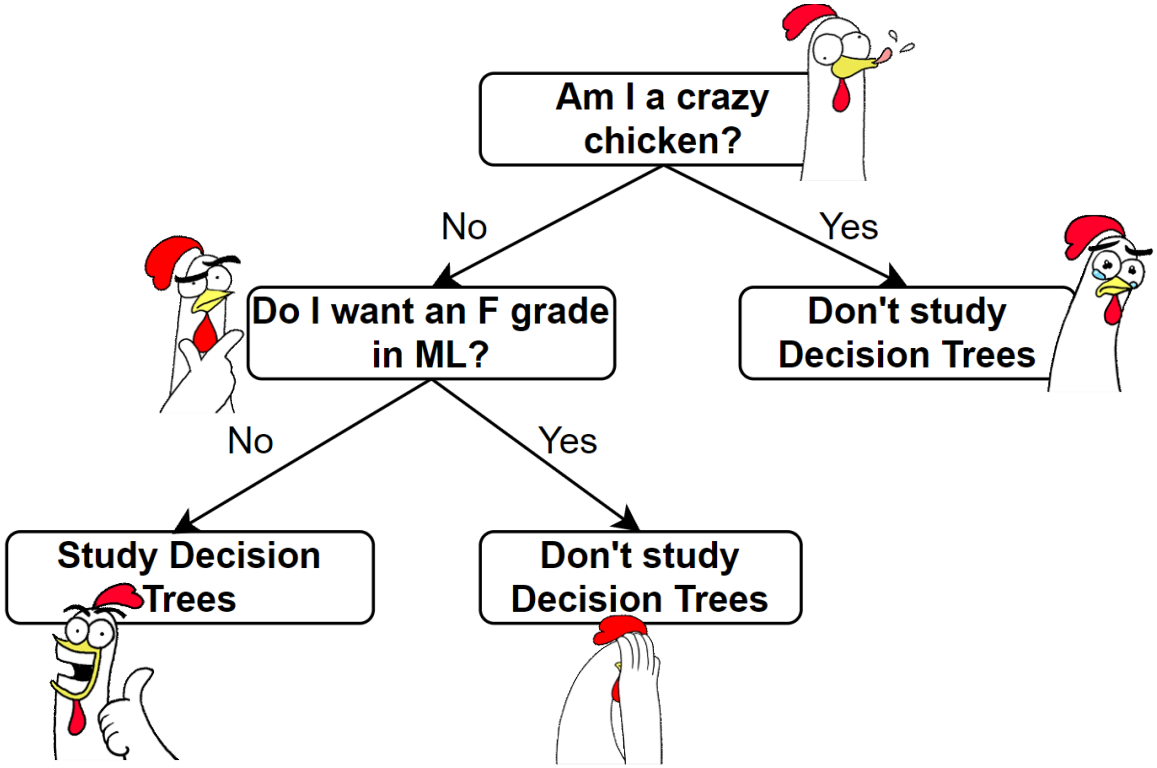
Agha Ali Raza

CS535/EE514 – Machine Learning

Sources

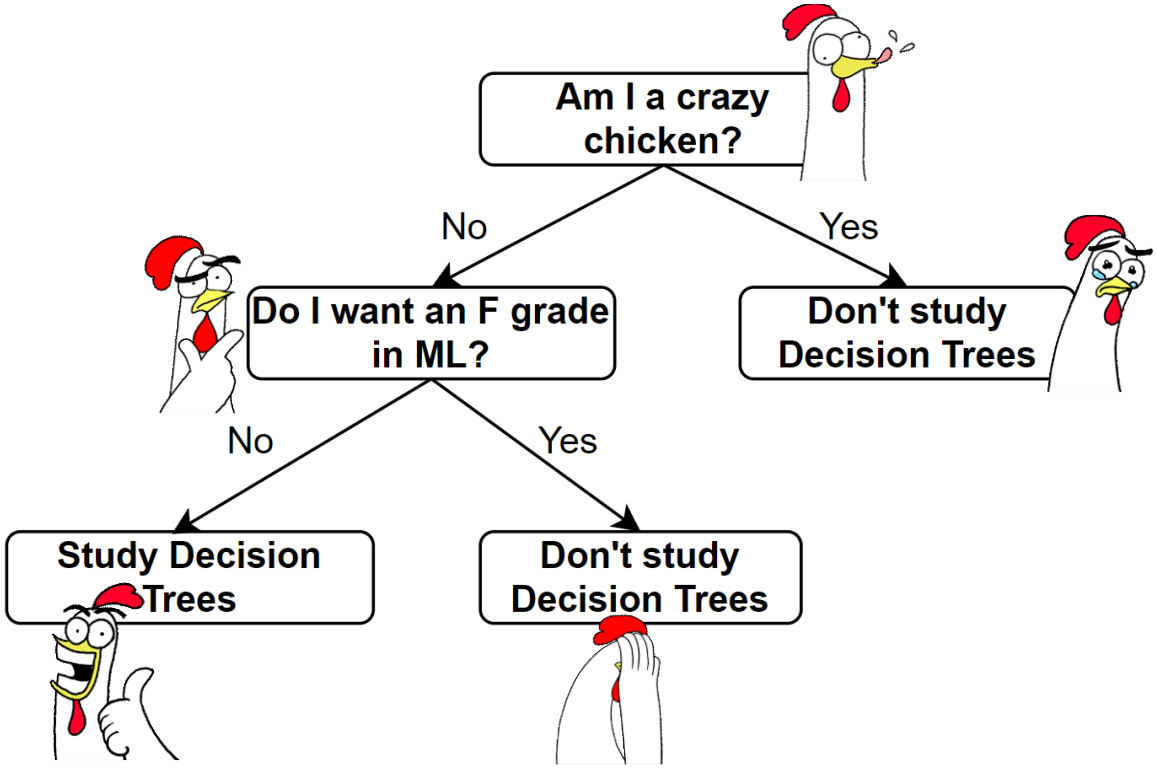
- **Decision Tree Learning,** Victor Levrenko, Professor at the University of Edinburgh, https://youtube.com/playlist?list=PLBv09BD7ez_4_UoYeGrzvqveIR_USBEKD
- **Machine Learning for Intelligent Systems**, Kilian Weinberger, Cornell, Lecture 16 (videos 28, 29), https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote17.html
- **StatQuest with Josh Starmer**
  - Regression Trees, Clearly Explained!!!, https://www.youtube.com/watch?v=g9c66TUylZ4
  - Entropy (for data science) Clearly Explained!!!, https://www.youtube.com/watch?v=YtebGVx-Fxw
  - Expected Values, Main Ideas!!!, https://www.youtube.com/watch?v=KLs_7b7SKi4
  - StatQuest: Decision Trees, https://www.youtube.com/watch?v=7VeUPuFGJHk
- **Information entropy | Journey into information theory | Computer Science | Khan Academy**, Khan Academy Labs, https://www.youtube.com/watch?v=2s3aJfRr9gE

Why do chickens learn decision trees?



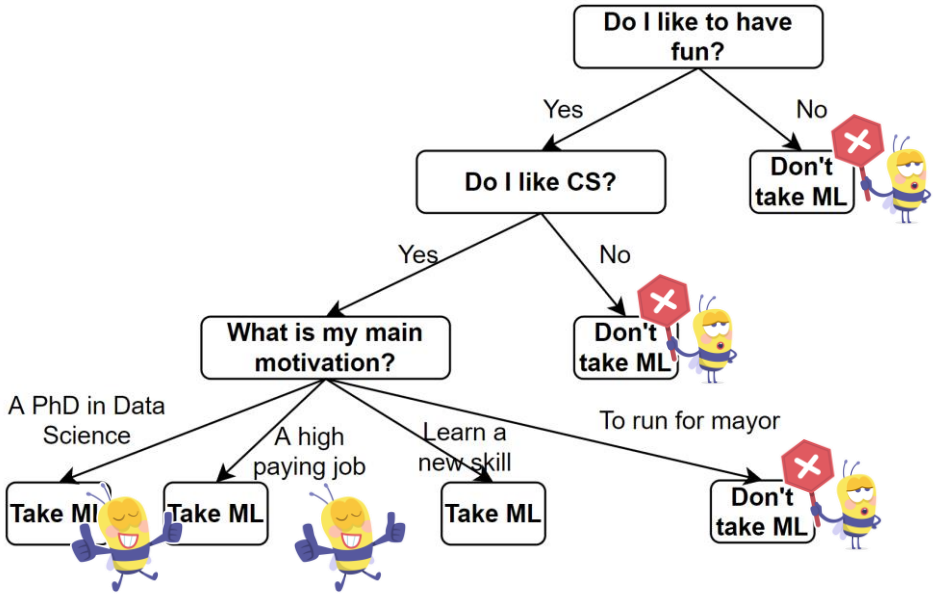
Because it was not crazy AND it did not want an F grade in ML!

Why do chickens learn decision trees?



Because it was not crazy AND it did not want an F grade in ML!

To be or not to be a (Machine Learner) Bee!



Take ML if you "like to have fun" AND "like CS" AND (want a PhD in DS OR a high paying job OR to learn a new skill) =

$$\text{Like Fun} \wedge \text{Like CS} \wedge (\text{PhD} \vee \text{Pay} \vee \text{Skill}) =$$
$$(\text{Like Fun} \wedge \text{Like CS} \wedge \mathit{PhD}) \vee (\text{Like Fun} \wedge \text{Like CS} \wedge \mathit{Pay}) \vee (\text{Like Fun} \wedge \text{Like CS} \wedge \mathit{Skill})$$

- Highly intuitive and explainable model
- Read the concise description of what makes an item positive off the tree
  - No black boxes
- Disjunctive Normal Form (DNF) – is a universal approximator!
  - The NOT operation can simply be added by asking a negative question e.g. "having a skill" vs "not having a skill"

Simple Representation

if $x_2 > 5$:

    if $x_1 < 2$:

        Class A

    else:

        Class B

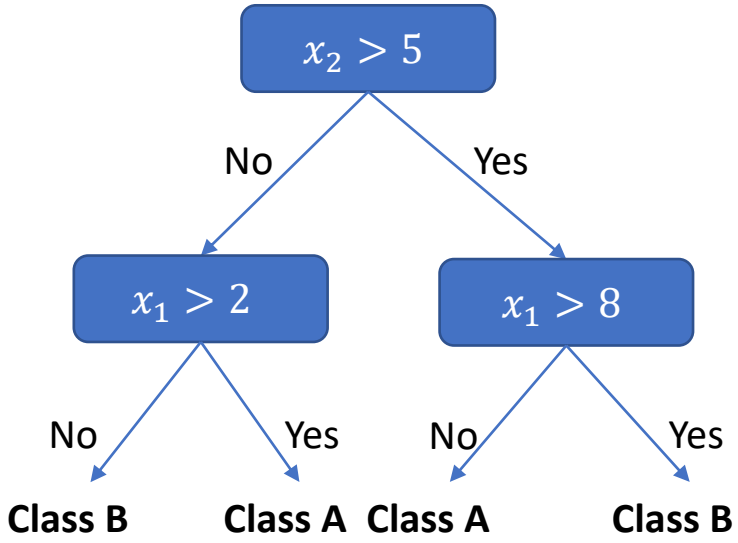
else:

    if $x_1 > 8$:

        Class B

    else:

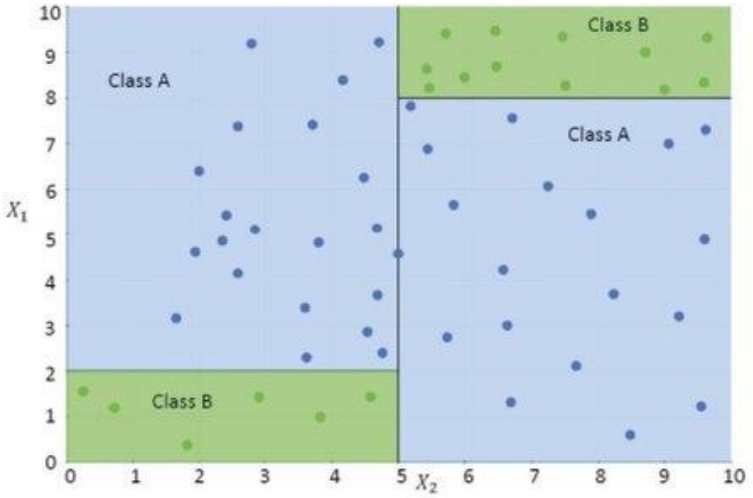
        Class A



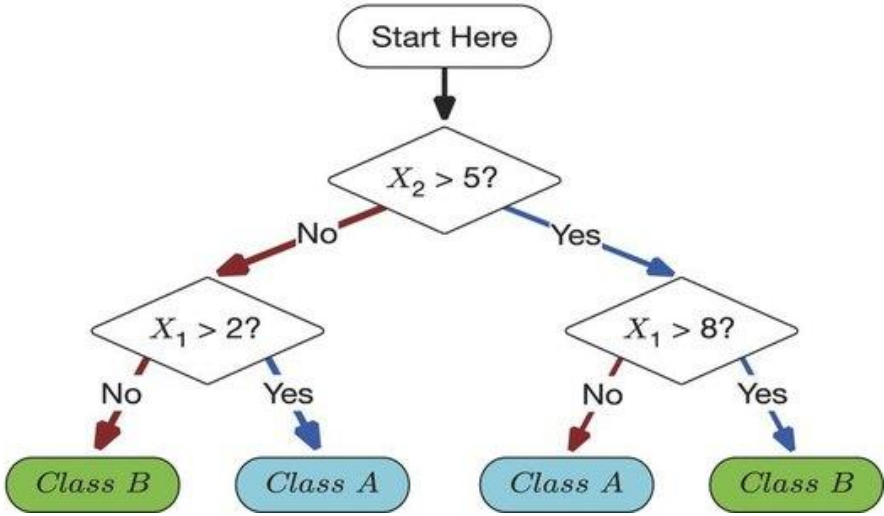
Partitioning of feature space – Real numbered attributes

- if $x_2 > 5$:
- if $x_1 < 2$:
- Class A
- else:
- Class B
- else:
- if $x_1 > 8$:
- Class B
- else:
- Class A

a)



b)

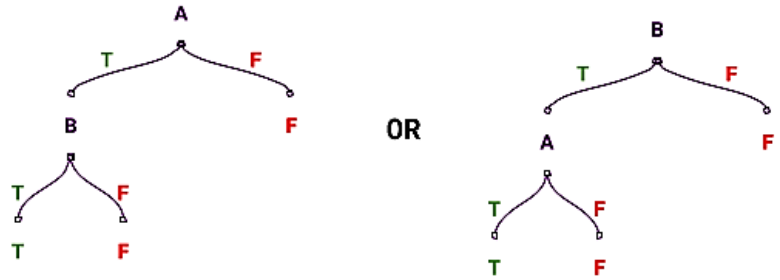


- For each split we create axis-aligned hyperplanes in n-Dimensions

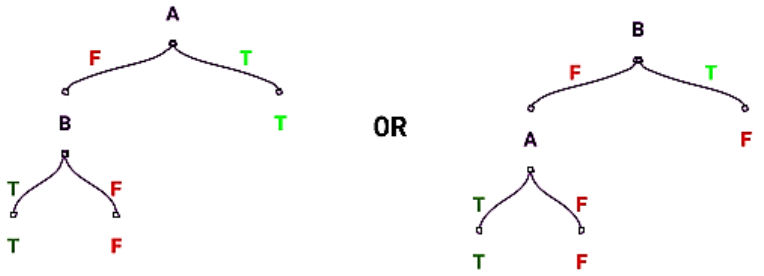
- For **real-valued attributes,** we need to define **thresholds**

Boolean functions

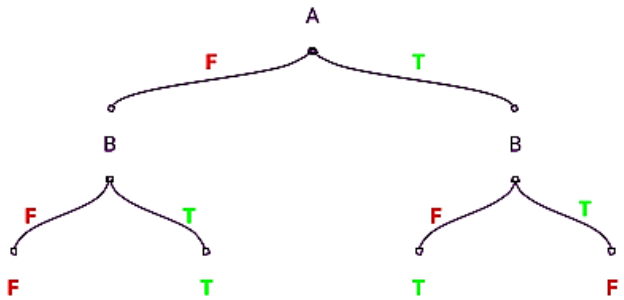
A	B	A AND B
F	F	F
F	T	F
T	F	F
T	T	T



A	B	A OR B
F	F	F
F	T	T
T	F	T
T	T	T

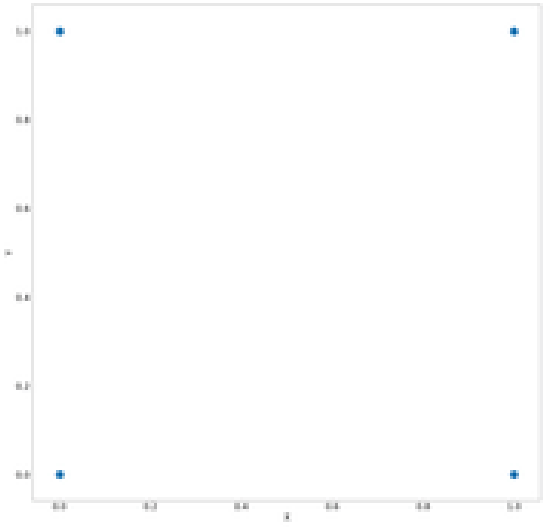


A	B	A XOR B
F	F	F
F	T	T
T	F	T
T	T	F



Boolean functions

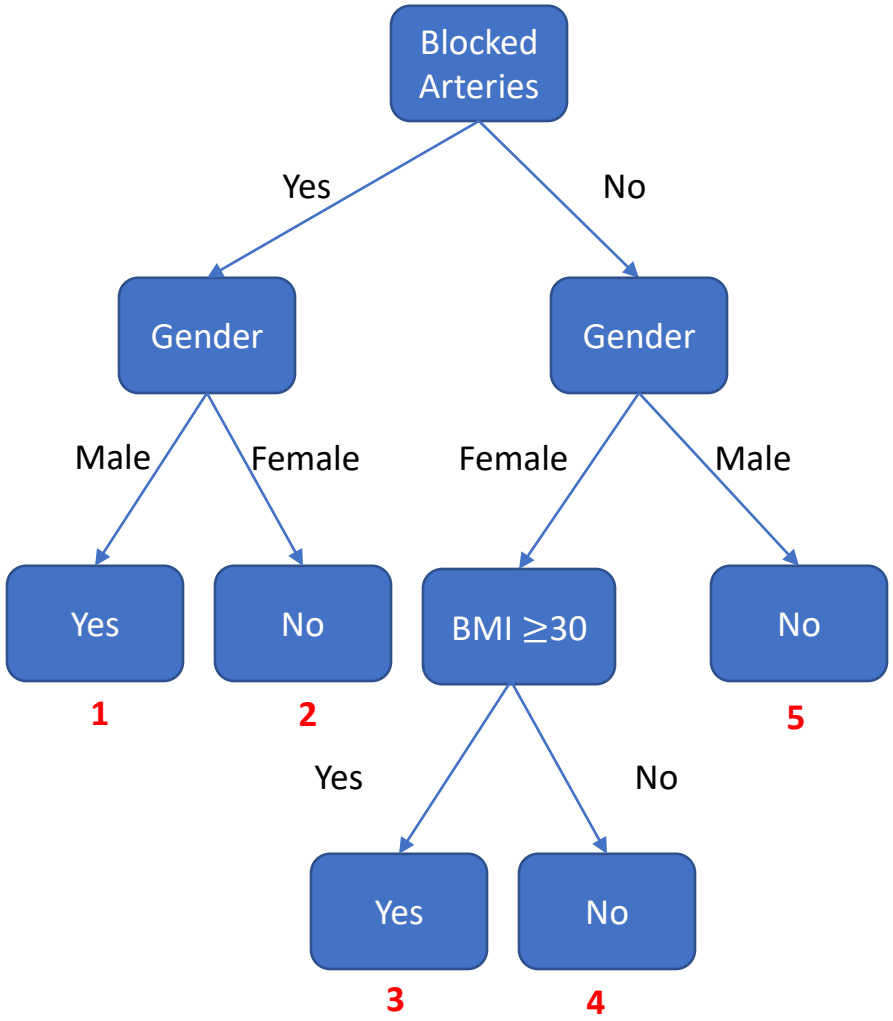
X	Y	X AND Y
0	0	0
0	1	0
1	0	0
1	1	1



Decision Trees for Classification and Regression

Age	Blocked Arteries	Height (inches)	Gender	BMI	Heart Disease	Threat (%)	
23	No	67	M	35	No	47	5
45	Yes	61	F	34	No	45	2
37	Yes	68	M	23	Yes	60	1
56	Yes	65	M	36	Yes	78	1
75	No	67	F	37	Yes	85	3
63	No	69	F	39	Yes	90	3
72	Yes	64	F	22	No	20	2
21	No	62	F	23	No	23	4
67	Yes	72	M	19	Yes	55	1
49	No	70	M	40	No	49	5

Age	Blocked Arteries	Height (inches)	Gender	BMI	Heart Disease	Threat (%)
25	No	63	M	23		
34	Yes	70	M	33		
50	No	65	F	33		



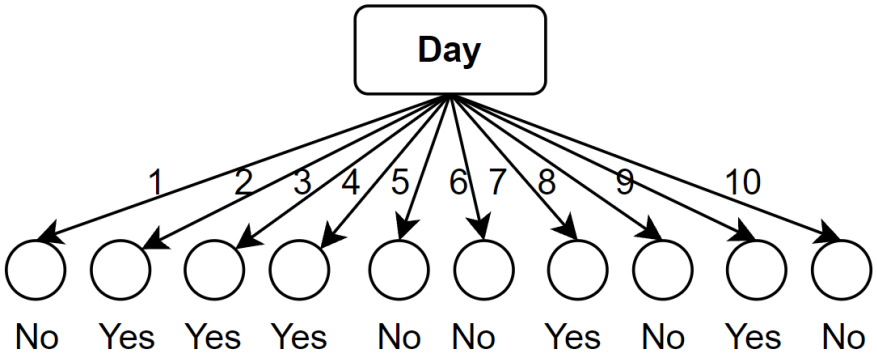
Might be incorrect as the BMI is low. So we need to grow a separate tree for regression.

Voting in case of impure leaves.

How about this tree? Short, sweet and perfect?

Day	Weather	Temperature	Humidity	Wind	Play?
1	Sunny	Hot	High	Weak	No
2	Cloudy	Hot	High	Weak	Yes
3	Sunny	Mild	Normal	Strong	Yes
4	Cloudy	Mild	High	Strong	Yes
5	Rainy	Mild	High	Strong	No
6	Rainy	Cool	Normal	Strong	No
7	Rainy	Mild	High	Weak	Yes
8	Sunny	Hot	High	Strong	No
9	Cloudy	Hot	Normal	Weak	Yes
10	Rainy	Mild	High	Strong	No

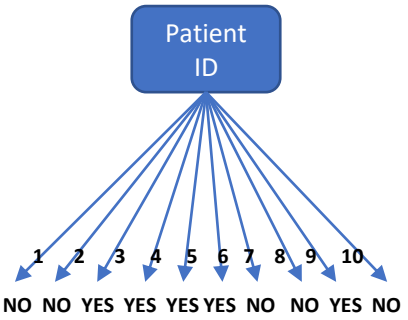
	Weather	Temperature	Humidity	Wind	Play?
13	Cloudy	Mild	High	Strong	?
19	Rainy	Mild	Normal	Strong	?



- This is overfitting!
- This tree is a perfect classifier for the training data but will never work for any new data
- We would need ways to prevent such trees from happening
- We do so by discouraging leaves with a very small number of instances

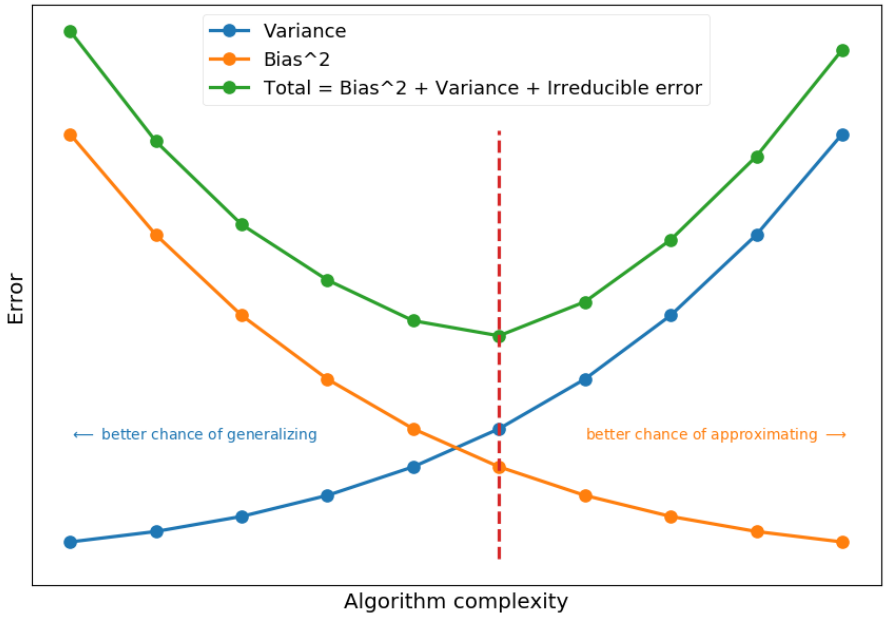
Good Trees – Bad Trees

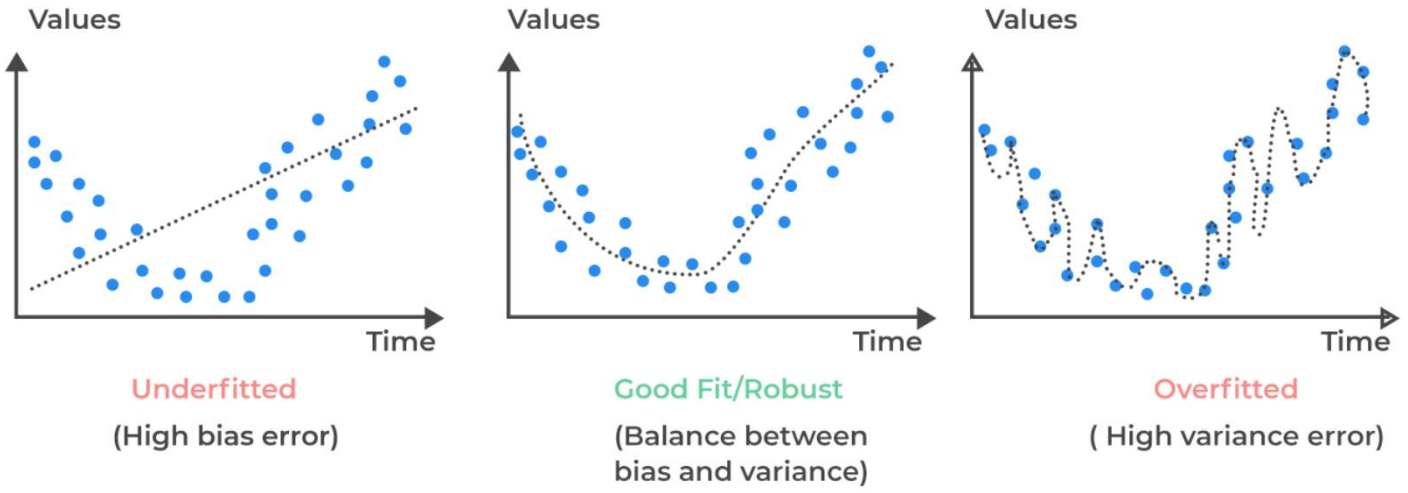
Age	Patient ID	Blocked Arteries	Height (inches)	Gender	BMI	Heart Disease	Threat (%)
23	1	No	67	M	35	No	47
45	2	Yes	61	F	34	No	45
37	3	Yes	68	M	23	Yes	60
56	4	Yes	65	M	36	Yes	78
75	5	No	67	F	37	Yes	85
63	6	No	69	F	39	Yes	90
72	7	Yes	64	F	22	No	20
21	8	No	62	F	23	No	23
67	9	Yes	72	M	19	Yes	55
49	10	No	70	M	40	No	49



Bias and Variance

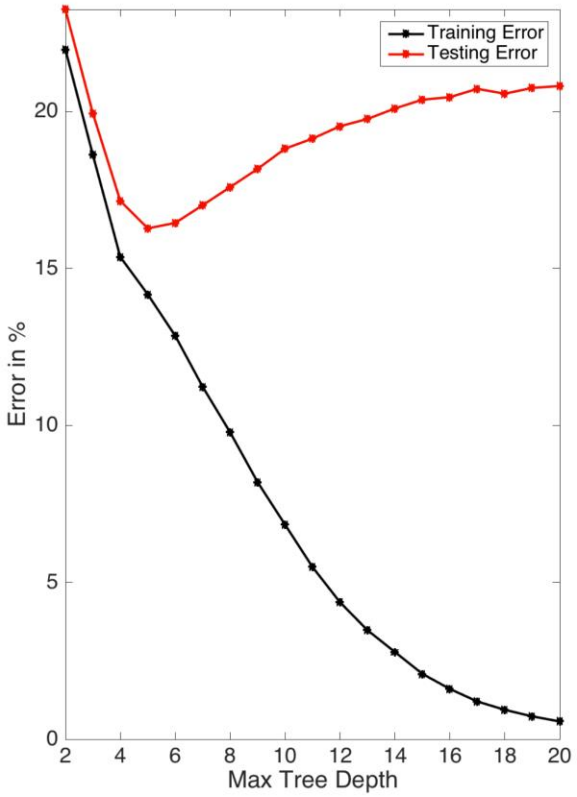
- **Bias** is the error that comes from wrong model assumptions, which do not allow an algorithm to learn all patterns from training data.
- **Variance** is the error that comes from model sensitivity to features specific to the training data (e.g., noise).
- **High bias** usually leads to **underfitting** and **high variance** usually leads to **overfitting**.
- Total error also depends on irreducible error (noise that can not be reduced by algorithm)
- Our Ultimate goal is to minimize the total error



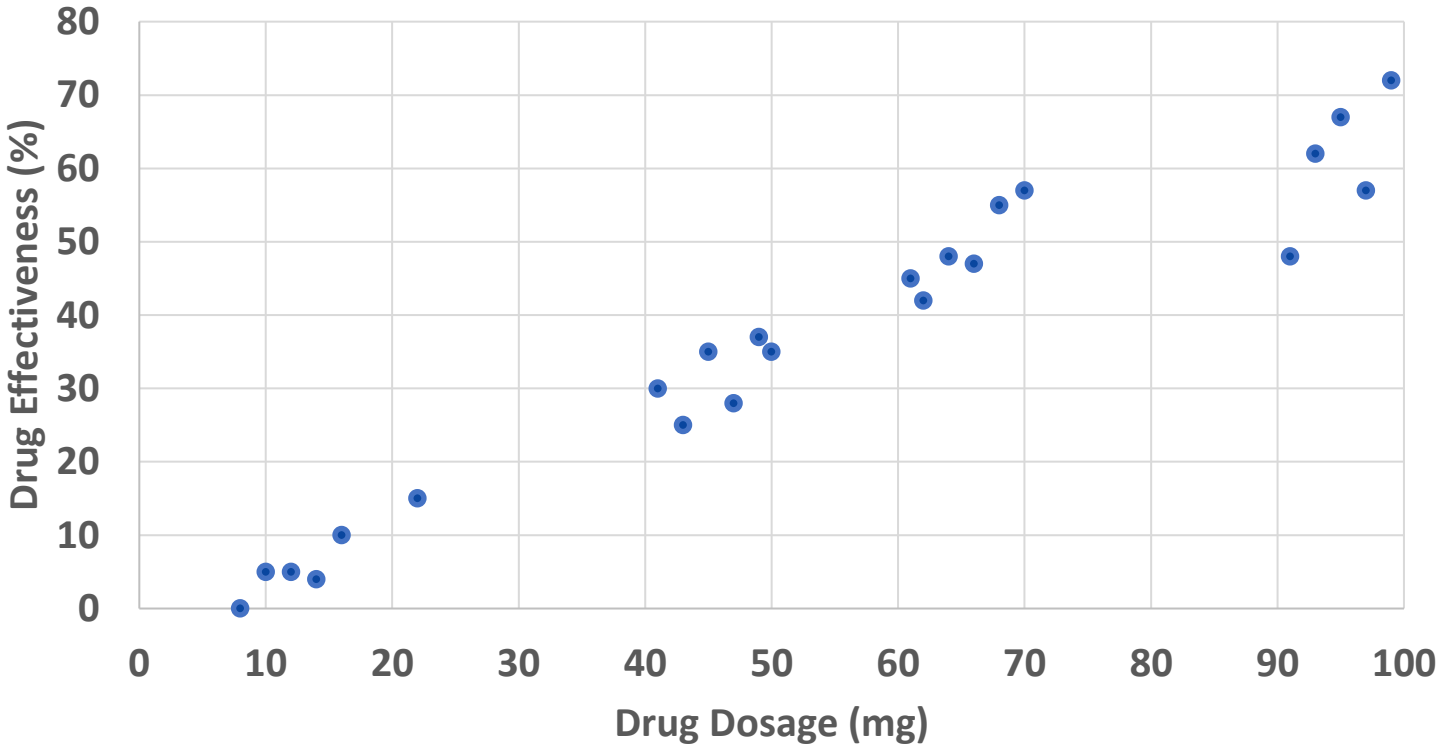


Bias and Variance – Height of Tree

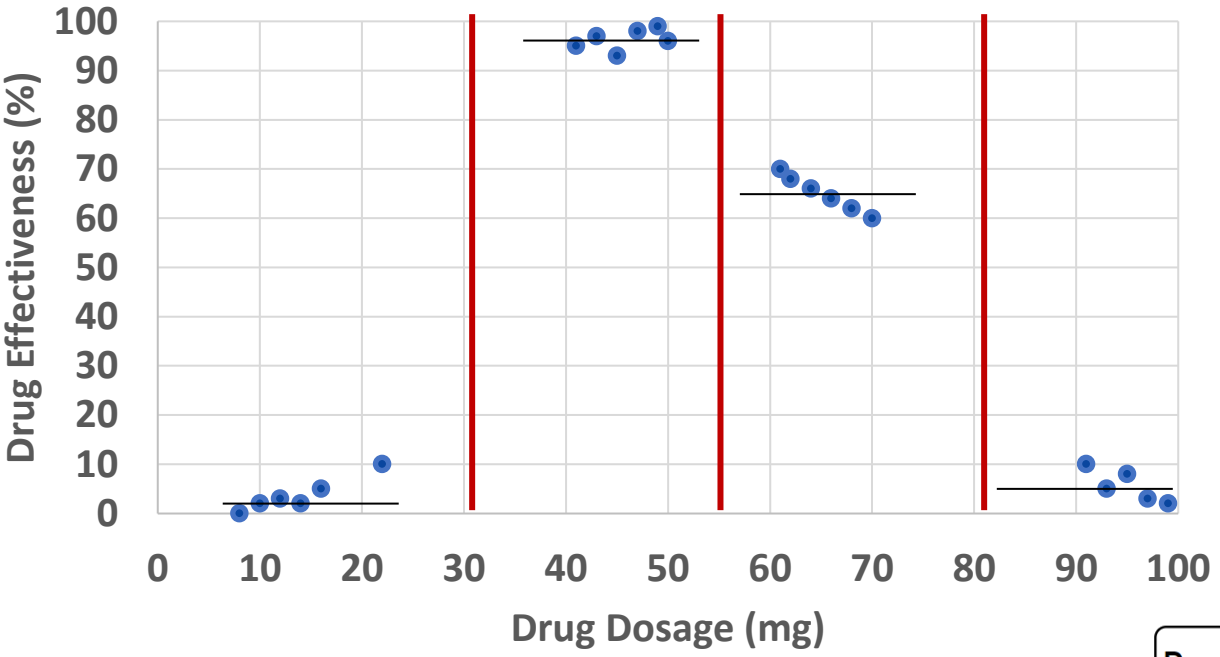
- Decision trees are sensitive to splits – small changes in training data may change a tree structure
- Shallow trees have high bias – underfitting – and low variance
- Deeper trees have high variance – overfitting – and low bias
  - We can always add a levels to perfectly fit any training data and even model the noise
    - Unless the dataset is inconsistent – instances having different labels for the same inputs
    - This leads to very high variance
    - The leaves may just have singletons left in them
- **Solutions (to be discussed later):**
  - Reduce **variance** using **Bagging** (Random Forests)
  - Reduce **bias** using **Boosting** (ada boost, gradient boosted trees)



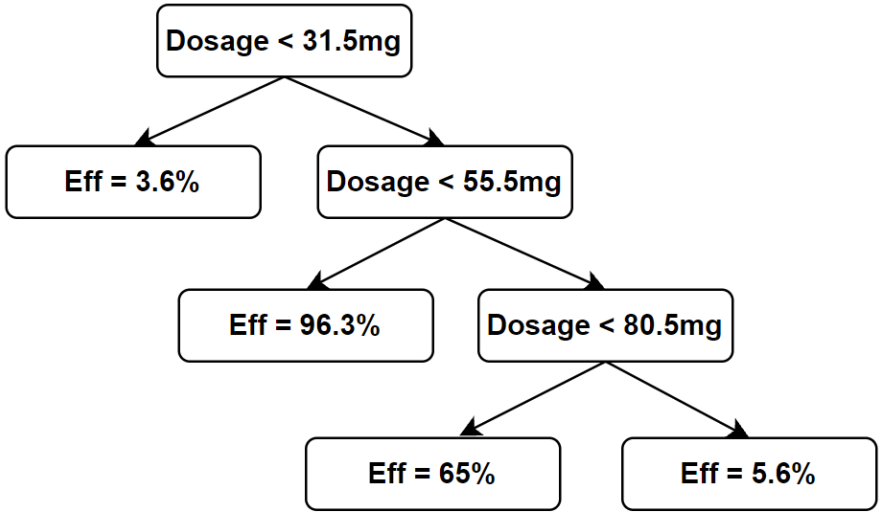
Regression using decision trees



Regression using decision trees



New datapoint: dosage = 70mg

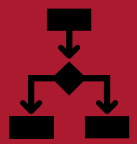


LUMS

CENTER FOR SPEECH AND LANGUAGE TECHNOLOGIES

Training a Decision Tree





Agha Ali Raza

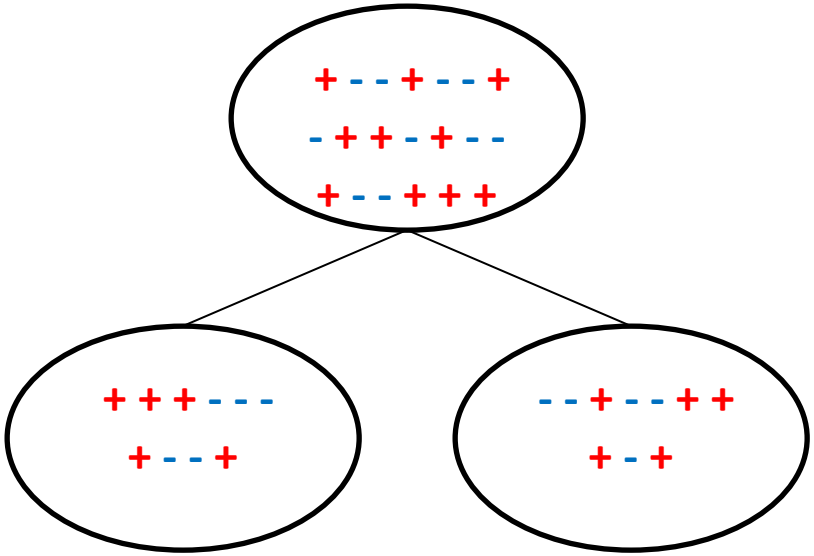
CS535/EE514 – Machine Learning

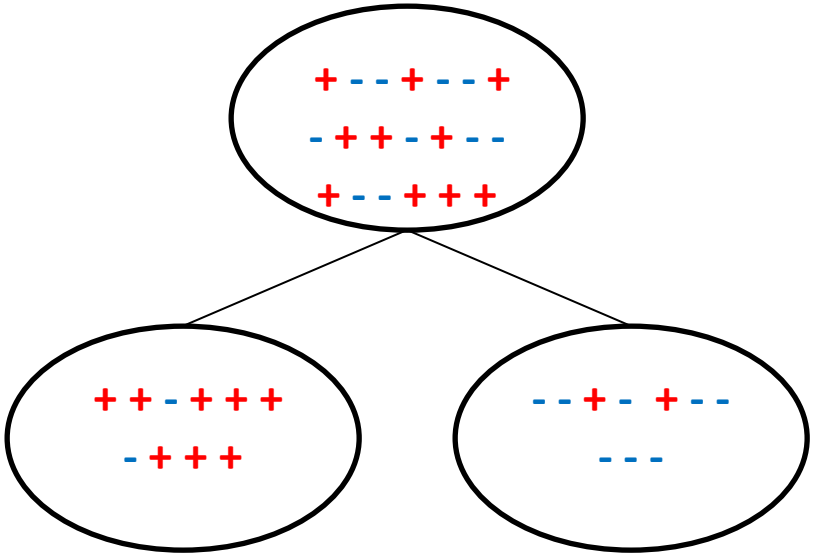
Sources

- **Decision Tree Learning,** Victor Levrenko, Professor at the University of Edinburgh, https://youtube.com/playlist?list=PLBv09BD7ez_4_UoYeGrzvqveIR_USBEKD
- **Machine Learning for Intelligent Systems**, Kilian Weinberger, Cornell, Lecture 16 (videos 28, 29), https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote17.html
- **StatQuest with Josh Starmer**
  - Regression Trees, Clearly Explained!!!, https://www.youtube.com/watch?v=g9c66TUylZ4
  - Entropy (for data science) Clearly Explained!!!, https://www.youtube.com/watch?v=YtebGVx-Fxw
  - Expected Values, Main Ideas!!!, https://www.youtube.com/watch?v=KLs_7b7SKi4
  - StatQuest: Decision Trees, https://www.youtube.com/watch?v=7VeUPuFGJHk
- **Information entropy | Journey into information theory | Computer Science | Khan Academy**, Khan Academy Labs, https://www.youtube.com/watch?v=2s3aJfRr9gE

How do we grow the tree automatically?

- We have seen pre-grown trees and have some idea of manually creating them by trial-and-error
  - Is there a more systematic, quantifiable approach towards choice of features for creating the splits and growing the tree step-by-step?
- Which one of the following is the better split?
  - Why?

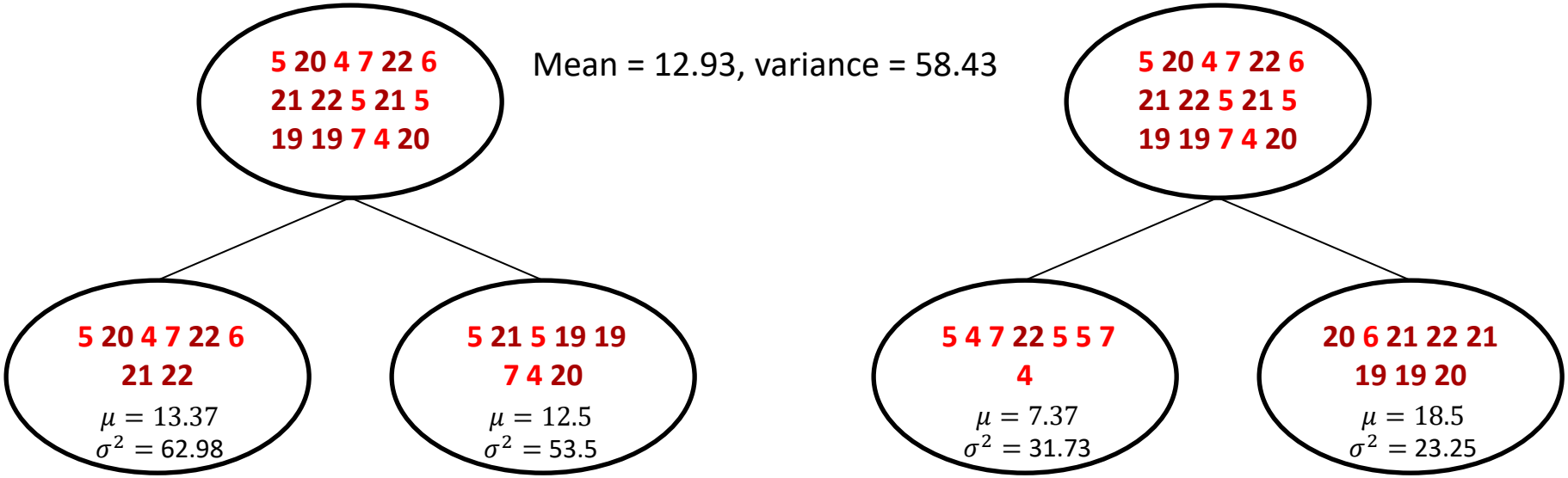




- We prefer splits that lead to “pure” outcomes!
- We need measures for “purity” or “impurity”

How do we grow the tree automatically?

- We have seen pre-grown trees and have some idea of manually creating them by trial-and-error
  - Is there a more systematic, quantifiable approach towards choice of features for creating the splits and growing the tree step-by-step?
- Which one of the following is the better split?
  - Why?



- We prefer splits that lead to "pure" outcomes!
- We need measures for "purity" or "impurity"

Training a decision tree – The ID3 and CaRT Algorithm

Iterative Dichotomizer (ID3), Classification and Regression Trees

Age	Blocked Arteries	Height (inches)	Gender	BMI	Heart Disease	Threat (%)
23	No	67	M	35	No	47
45	Yes	61	F	34	No	45
37	Yes	68	M	23	Yes	60
56	Yes	65	M	36	Yes	78
75	No	67	F	37	Yes	85
63	No	69	F	39	Yes	90
72	Yes	64	F	22	No	20
21	No	62	F	23	No	23
67	Yes	72	M	19	Yes	55
49	No	70	M	40	No	49

- Split (node, {instances}, {attributes})
  1. A ← the best attribute for splitting the examples
     - In case of real-valued attributes, scan across all thresholds ($t \in T$) for each attribute
  2. Decision attribute for this node ← A (threshold ← T)
  3. For each value of A, create a new child node
     - Create two children around the threshold for real-valued attributes
  4. Split training examples among child nodes
  5. For each child node/subset
     1. If subset is pure (or some other stopping criterion is met), STOP
     2. Else: split(child node, {subset of instances})

- Ross Quinlan, CS (ID3: 1986, C4.5: 1993)
  - Uses entropy as the impurity function
  - Meant primarily for categorical attributes and for classification
- Breimanetal, Statistics (CaRT: 1984)
  - Uses Gini impurity
  - Meant for classification and regression

https://www.youtube.com/watch?v=9DuopoXtemc&list=PLBv09BD7ez_4_UoYeGrzvqveIR_USBEKD&index=3

Measuring "Impurity"

Data: $S = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$, $y_i \in \{1, \dots c\}$, where c is the number of classes

- Let $S_k \subseteq$ S, where $S_k = \{(x, y) \in S : y = k\}$ i.e. all inputs with label k.
- $S = S_1 \cup \cdots \cup S_c$
- $p_k = \frac{|S_k|}{|S|}$, fraction of inputs in S with label k
- **Classification**
  - Gini Impurity

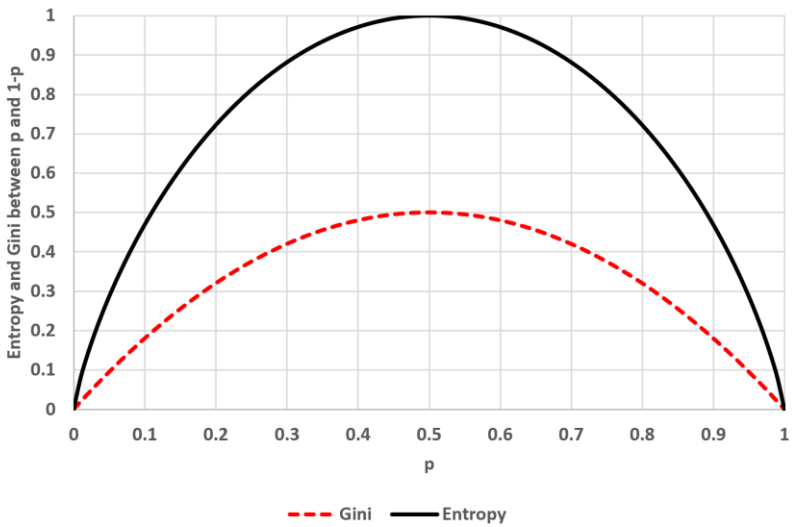
$$\boldsymbol{G(S) = \sum_{k=1}^{c} p_k(1 - p_k)}$$

  - Entropy

$$\boldsymbol{H(S) = \sum_{k=1}^{c} p_k \log_2(\frac{1}{p_k})}$$

$$H(S) = \sum_{k=1}^{c} p_k \left[log_2(1) - \log_2(p_k)\right]$$

$$\boldsymbol{H(S) = -\sum_{k=1}^{c} p_k \log_2(p_k)}$$

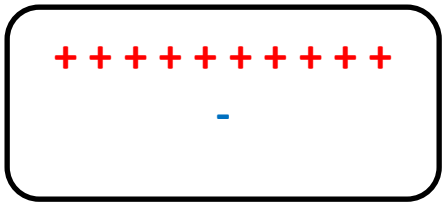


https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote17.html

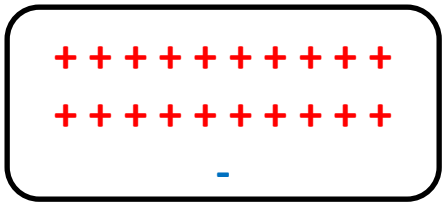
Gini and Entropy

$$G(S) = \sum_{k=1}^{c} p_k(1 - p_k), \qquad H(S) = \sum_{k=1}^{c} p_k \log_2\left(\frac{1}{p_k}\right)$$

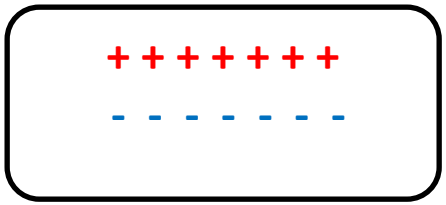
$p_+ = \frac{10}{11} = 0.90, \quad p_- = \frac{1}{11} = 0.09, \quad Gini = 0.16, \quad H = 0.43$



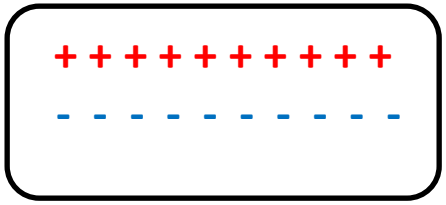
$p_+ = \frac{20}{21} = 0.95, \quad p_- = \frac{1}{21} = 0.047, \quad Gini = 0.09, \quad H = 0.28$



$p_+ = \frac{7}{14} = 0.5, \quad p_- = \frac{7}{14} = 0.5, \quad Gini = 0.5, \quad H = 1.0$



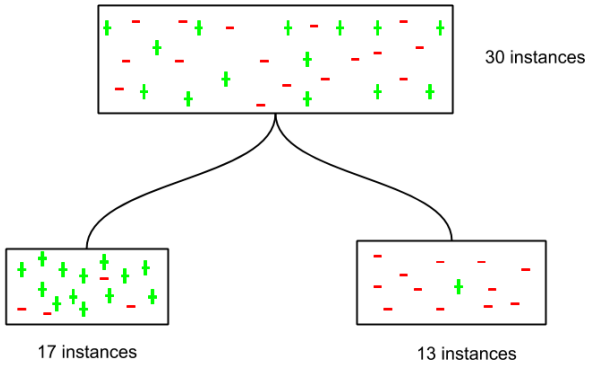
$p_+ = \frac{10}{20} = 0.5, \quad p_- = \frac{10}{20} = 0.5, \quad Gini = 0.5, \quad H = 1.0$



The reduction in Gini impurity due to a split

Gini impurity of a split:

$$G(S) = \frac{|S_L|}{|S|} G(S_L) + \frac{|S_R|}{|S|} G(S_R)$$



$$\frac{17}{30} G(S_L) + \frac{13}{30} G(S_R)$$

Where,

- $(S = S_L \cup S_R)$
- $S_L \cap S_R = \emptyset$
- $\frac{|S_L|}{|S|}$, fraction of inputs in left subtree
- $\frac{|S_R|}{|S|}$, fraction of inputs in right subtree
- The weighing allows giving more weight to larger subsets

- Cycle through splits using all unused features and choose the one that results in the **lowest impurity** after the split.

Information Gain of a Split

Information gain of a split:

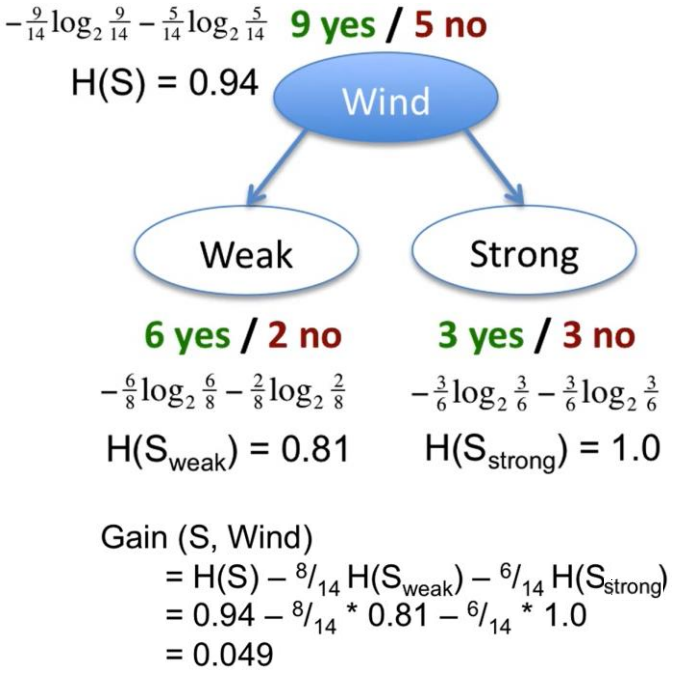
$$\textbf{Information Gain} = \textbf{Entropy}(\textbf{parent}) - [\textbf{weighted mean Entropy}(\textbf{children})]$$

$$\boldsymbol{Gain(S,A) = H(S) - \left(\frac{|S_L|}{|S|} H\,(S_L) + \frac{|S_R|}{|S|} H(S_R)\right)}$$

Where,

- A is the attribute being used in the split
- $(S = S_L \cup S_R)$
- $S_L \cap S_R = \emptyset$
- $\frac{|S_L|}{|S|}$, fraction of inputs in left subtree
- $\frac{|S_R|}{|S|}$, fraction of inputs in right subtree
- The weighing allows giving more weight to larger subsets

- Cycle through splits using all unused features
- Choose the feature that **maximizes** the gain
- This is also the **Mutual Information** between the attribute A and the class labels of S



Victor Lavrenko https://www.youtube.com/watch?v=2k-ZlWwxoCM&list=PLBv09BD7ez_4_UoYeGrzvqveIR_USBEKD&index=7

Gini and Entropy – Multiclass

#class1	#class2	#class3	n	p_{class1}	p_{class2}	p_{class3}	Gini	Entropy
10	1	1	12	0.83	0.08	0.08	0.29	0.82
20	1	1	22	0.91	0.05	0.05	0.17	0.53
20	10	1	31	0.65	0.32	0.03	0.48	1.09
20	10	10	40	0.50	0.25	0.25	0.63	1.50
10	10	10	30	0.33	0.33	0.33	0.67	1.58
20	20	20	60	0.33	0.33	0.33	0.67	1.58
100	1	1	102	0.98	0.01	0.01	0.04	0.16

Measuring "Impurity" for Regression

Average squared difference from average label (variance).

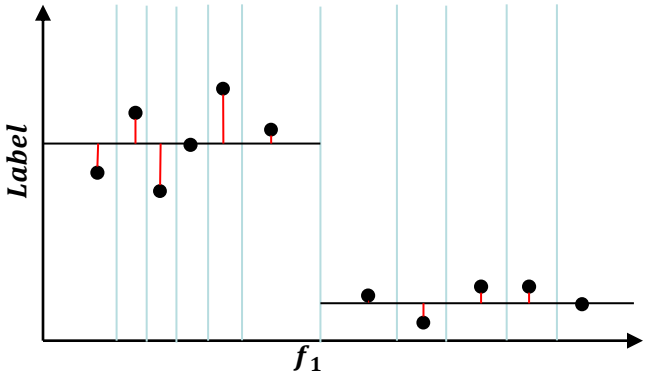
- For the set S:

$$L(S) = \frac{1}{|S|} \sum_{(x,y) \in S} (y - \bar{y}_S)^2$$

$$where\ \bar{y}_S = \frac{1}{|S|} \sum_{(x,y) \in S} y$$

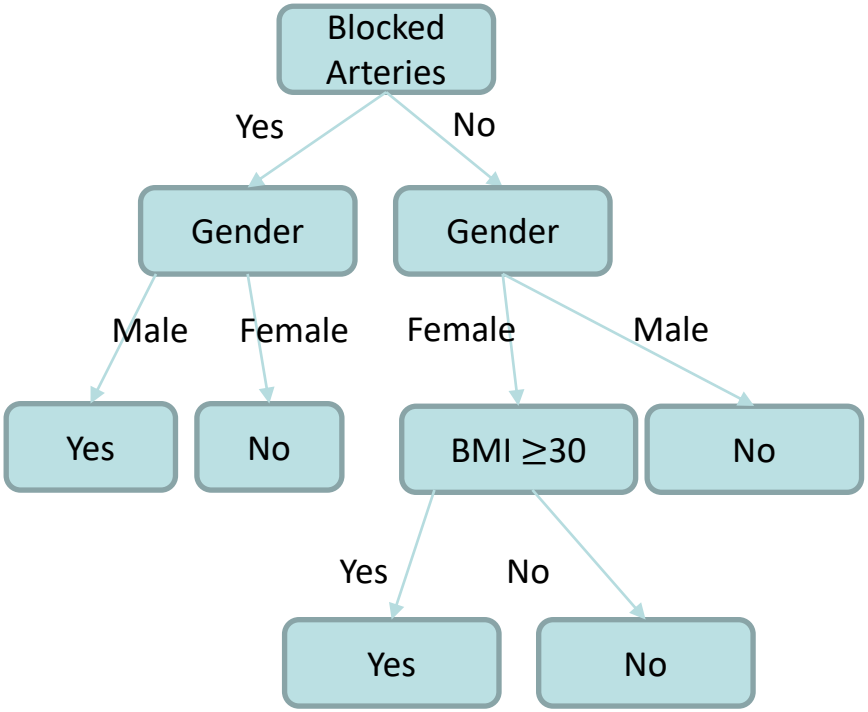
- Weighted mean variance of a split:

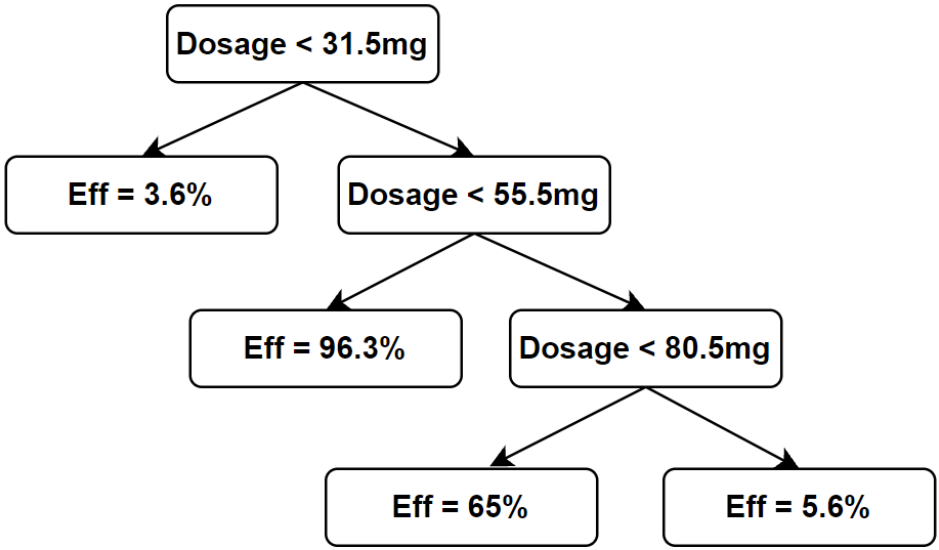
$$L(S) = \frac{|S_L|}{|S|} L(S_L) + \frac{|S_R|}{|S|} L(S_R)$$



Real-valued Attributes

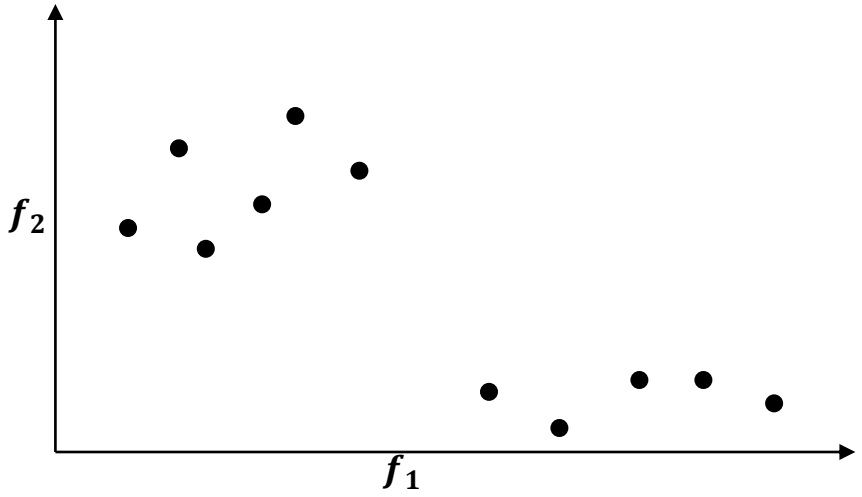
Repeating attributes – Real vs. categorical



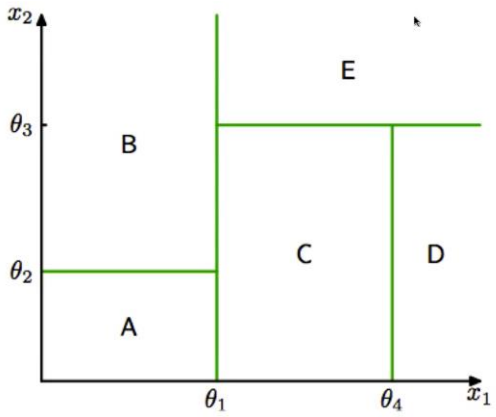


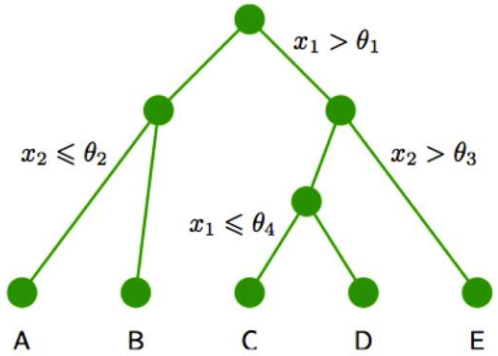
Real-valued attributes

- Optimizing thresholds
  - The naïve and systemic (greedy) methods



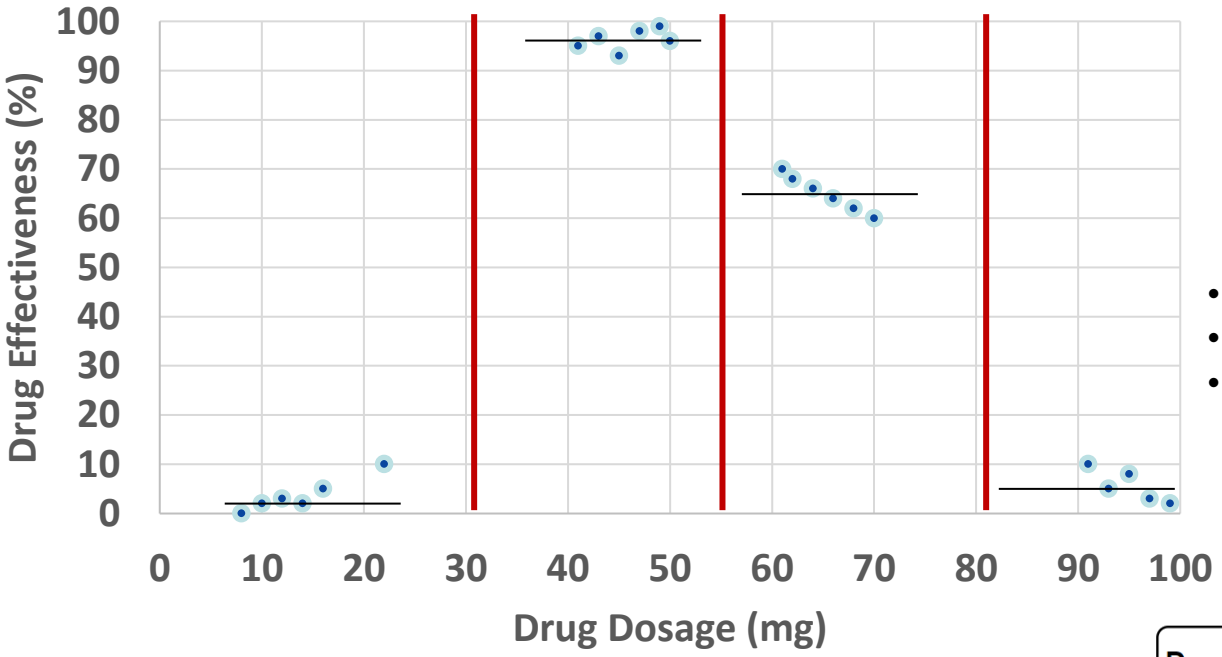
- Repeating attributes
  - Real vs. categorical





IAML7.11 Decision trees and real-valued data, ttps://www.youtube.com/watch?v=N08cHUKxENE&list=PLBv09BD7ez_4_UoYeGrzvqveIR_USBEKD&index=11

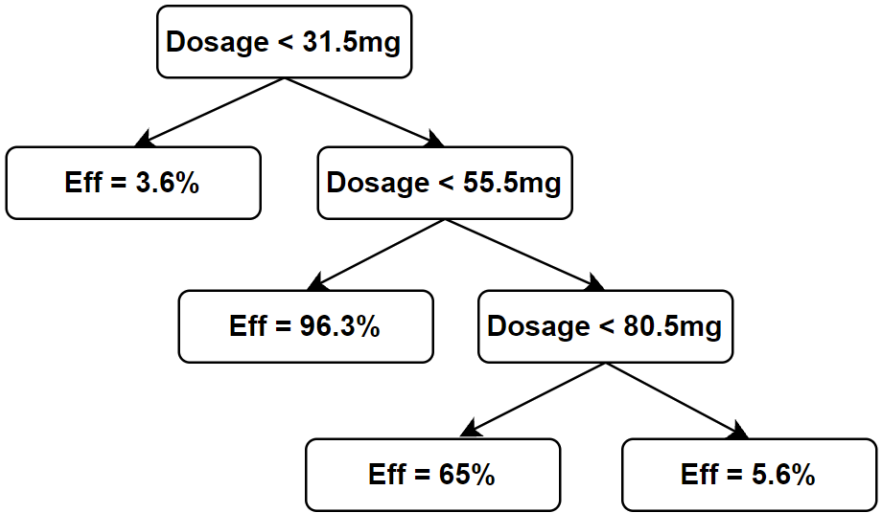
Regression using decision trees



$$L(S) = \frac{1}{|S|} \sum_{(x,y)\in S} (y - \bar{y}_S)^2 ,$$

$$where\ \bar{y}_S = \frac{1}{|S|} \sum_{(x,y)\in S} y$$

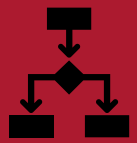
- How to split?
- When to stop?
- What do we do in case of multiple attributes?
  - E.g., dosage, age, bio gender



LUMS

CENTER FOR SPEECH AND LANGUAGE TECHNOLOGIES

DT: Strengths and Weaknesses



Agha Ali Raza

CS535/EE514 – Machine Learning

Sources

- **Decision Tree Learning,** Victor Levrenko, Professor at the University of Edinburgh, https://youtube.com/playlist?list=PLBv09BD7ez_4_UoYeGrzvqveIR_USBEKD
- **Machine Learning for Intelligent Systems**, Kilian Weinberger, Cornell, Lecture 16 (videos 28, 29), https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote17.html
- **StatQuest with Josh Starmer**
  - Regression Trees, Clearly Explained!!!, https://www.youtube.com/watch?v=g9c66TUylZ4
  - Entropy (for data science) Clearly Explained!!!, https://www.youtube.com/watch?v=YtebGVx-Fxw
  - Expected Values, Main Ideas!!!, https://www.youtube.com/watch?v=KLs_7b7SKi4
  - StatQuest: Decision Trees, https://www.youtube.com/watch?v=7VeUPuFGJHk
- **Information entropy | Journey into information theory | Computer Science | Khan Academy**, Khan Academy Labs, https://www.youtube.com/watch?v=2s3aJfRr9gE

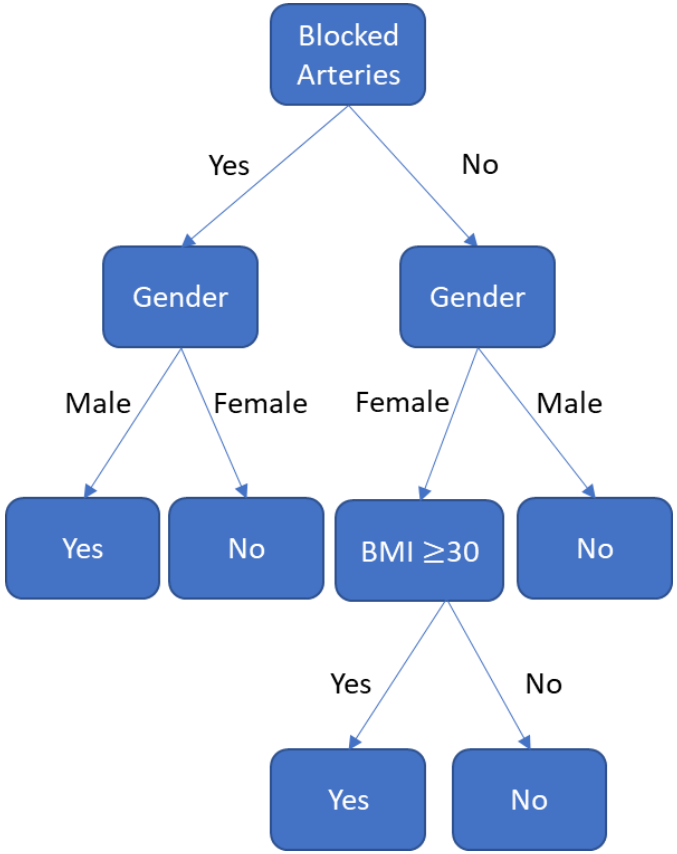
Strengths and Weaknesses

- Several nice advantages (esp. over nearest neighbor algorithms):
  - Storage efficiency – instead of storing the complete dataset we only need to store the labels of the leaves (mostly pure) and the thresholds (if-else) – #nodes << D
  - Testing efficiency: Decision trees are very fast during test time, as test inputs simply need to traverse down the tree to a leaf (O(height))
    - The prediction is the mode – in case of classification – and the mean – in case of regression
  - Decision trees require no metric (and feature normalization or scaling) because the splits are based on feature thresholds and not distances.
  - Interpretable: Easy to explain and visualize – DNF
  - Can be used for both classification and regression
- Weaknesses
  - Decision trees are sensitive to splits – small changes in training data may change a tree structure
  - Finding a minimum size tree is NP-Hard!!
    - Exponentially many trees
    - Good News: We can approximate it very effectively with a greedy strategy. We keep splitting the data to minimize an impurity function that measures label purity amongst the children.

Automatic feature selection

- Feature selection occurs when we reach pure leaves without ever having used certain attributes
- Can easily handle irrelevant attributes (Gain = 0)

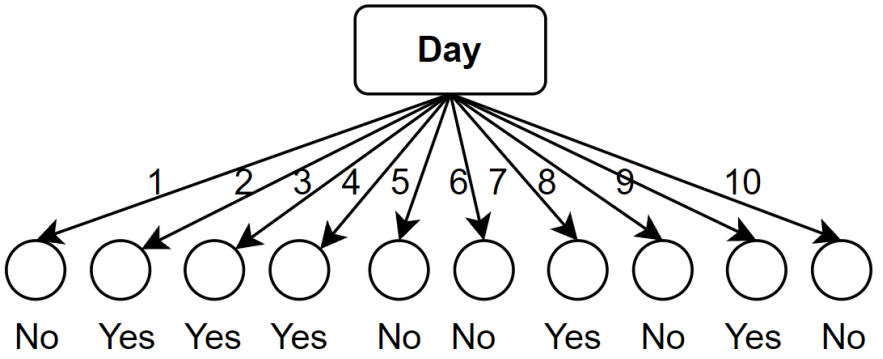
Age	Blocked Arteries	Height (inches)	Gender	BMI	Heart Disease	Threat (%)
23	No	67	M	35	No	47
45	Yes	61	F	34	No	45
37	Yes	68	M	23	Yes	60
56	Yes	65	M	36	Yes	78
75	No	67	F	37	Yes	85
63	No	69	F	39	Yes	90
72	Yes	64	F	22	No	20
21	No	62	F	23	No	23
67	Yes	72	M	19	Yes	55
49	No	70	M	40	No	49



Problems with Information gain: Preference towards attributes with several categories

Day	Weather	Temperature	Humidity	Wind	Play?
1	Sunny	Hot	High	Weak	No
2	Cloudy	Hot	High	Weak	Yes
3	Sunny	Mild	Normal	Strong	Yes
4	Cloudy	Mild	High	Strong	Yes
5	Rainy	Mild	High	Strong	No
6	Rainy	Cool	Normal	Strong	No
7	Rainy	Mild	High	Weak	Yes
8	Sunny	Hot	High	Strong	No
9	Cloudy	Hot	Normal	Weak	Yes
10	Rainy	Mild	High	Strong	No

	Weather	Temperature	Humidity	Wind	Play?
13	Cloudy	Mild	High	Strong	?
19	Rainy	Mild	Normal	Strong	?



- This is a very specific form of overfitting!
- This tree is a perfect classifier for the training data but will never work for any new data
- We would need ways to prevent such trees from happening
- We do so by discouraging leaves with very small number of instances

Information Gain Ratio

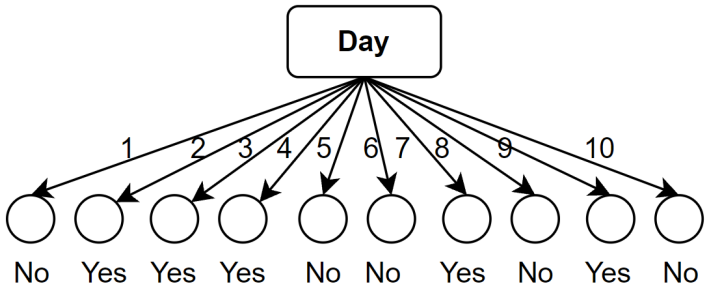
Information gain ratio

$$SplitEntropy(S,A) = -\sum_{v \in Values(A)} \frac{|S_v|}{|S|} \log_2 \left(\frac{|S_v|}{|S|} \right)$$

$$Gain\ Ratio(S,A) = \frac{Gain(S,A)}{SplitEntropy(S,A)}$$

- Lower values for fewer and bigger subsets
- Higher values for several tiny subsets
- E.g., S = 12
  - 2 equal splits, SE= 1.0
  - 3 equal splits, SE= 1.58
  - 4 equal splits, SE= 2.0
  - 6 equal splits, SE= 2.58
  - 12 equal splits, SE= 3.58
- If two attributes with different number of categories, have the same Entropy, Info Gain cannot differentiate them (Decision tree algorithm will select one of them randomly). In the same situation Gain Ratio, will favor attribute with less categories.
- Gain ratio leads to better generalization (less overfitting) of DT models and it is better to use Gain ratio in general.

Day	Weather	Temperature	Humidity	Wind	Play?
1	Sunny	Hot	High	Weak	No
2	Cloudy	Hot	High	Weak	Yes
3	Sunny	Mild	Normal	Strong	Yes
4	Cloudy	Mild	High	Strong	Yes
5	Rainy	Mild	High	Strong	No
6	Rainy	Cool	Normal	Strong	No
7	Rainy	Mild	High	Weak	Yes
8	Sunny	Hot	High	Strong	No
9	Cloudy	Hot	Normal	Weak	Yes
10	Rainy	Mild	High	Strong	No



Optimal split with all pure singletons.

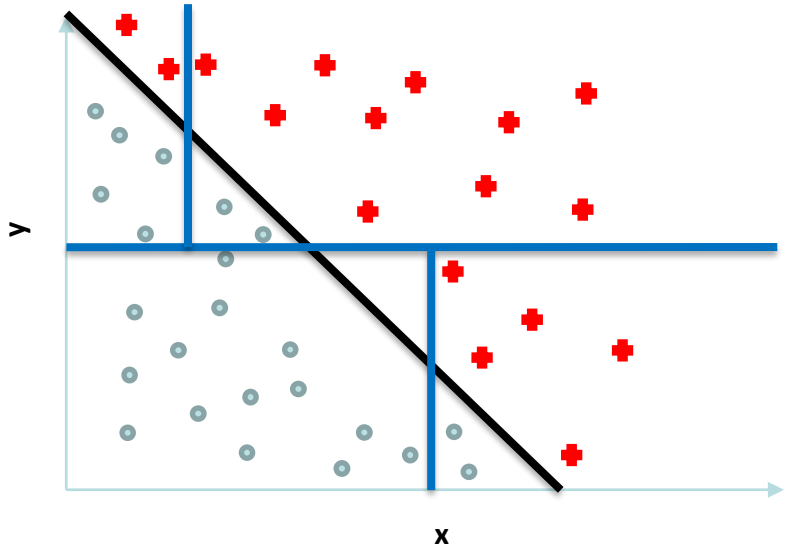
Missing data

Day	Weather	Temperature	Humidity	Wind	Play?
1	Sunny	Hot	High	Weak	No
2	Cloudy	Hot	High	Weak	Yes
3	Sunny	Mild	Normal	Strong	Yes
4	Cloudy	Mild	High	Strong	Yes
5	Rainy	?	High	Strong	No
6	Rainy	Cool	Normal	Strong	No
7	Rainy	Mild	High	Weak	Yes
8	Sunny	Hot	High	?	No
9	Cloudy	Hot	Normal	Weak	Yes
10	Rainy	Mild	High	Strong	No

- **Categorical attribute:** Vote amongst the datapoints with the same label
- **Numerical attribute:** Take the median of the datapoints with the same label

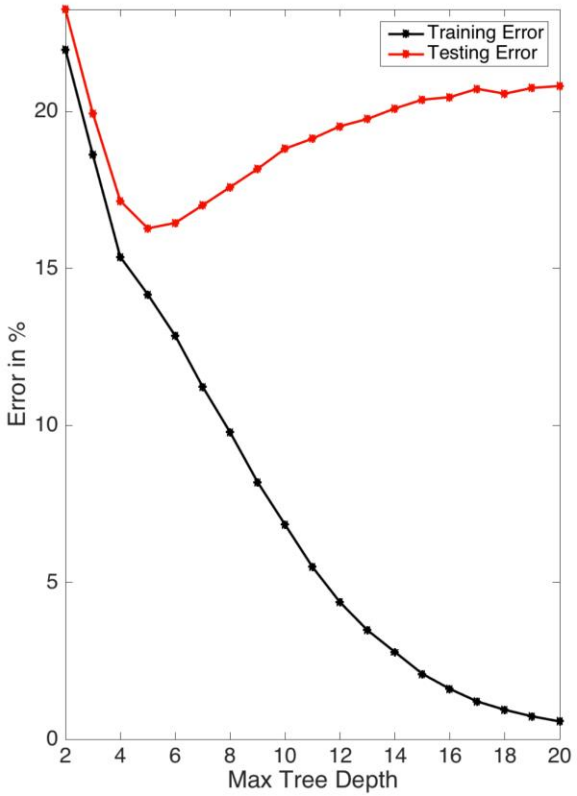
Weakness: Only Axis-aligned Splits

Such simple diagonal linear separations pose a challenge for DTs.



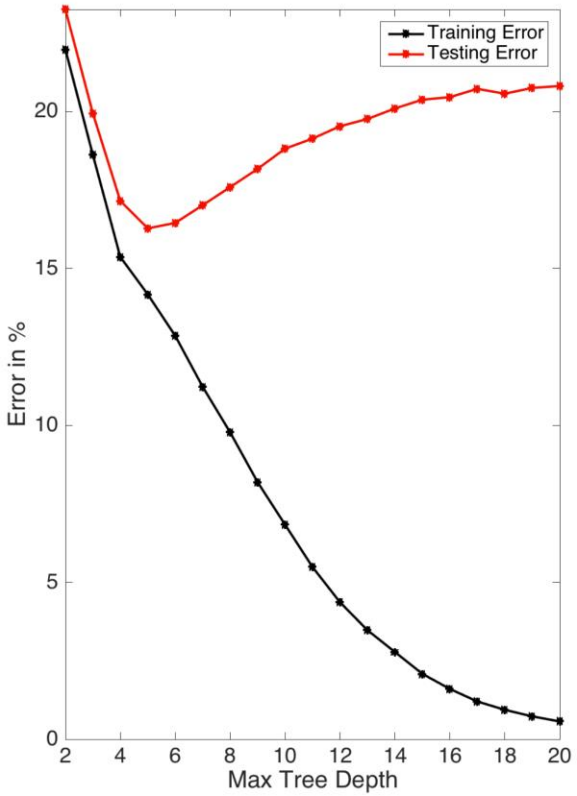
Decision Trees Easily Overfit

- We can always add levels to perfectly fit any training data and even model the noise
  - Unless the dataset is inconsistent – instances having different labels for the same inputs
  - This leads to very high variance
  - The leaves may just have singletons left in them – pure by definition
- **Solutions: Pruning**
  - **Pre-pruning or early stopping:** Stop growing the tree:
    - At a predefined height
    - At a predefined minimum number of datapoints per leaf
    - At a minimum threshold for information gain
    - As soon as validation error starts climbing
  - Considered more efficient as trees remain small
  - **Problem:** The horizon effect - Early stopping may underfit by stopping too early. The current split may be of little benefit, but having made it, subsequent splits more significantly reduce the error.



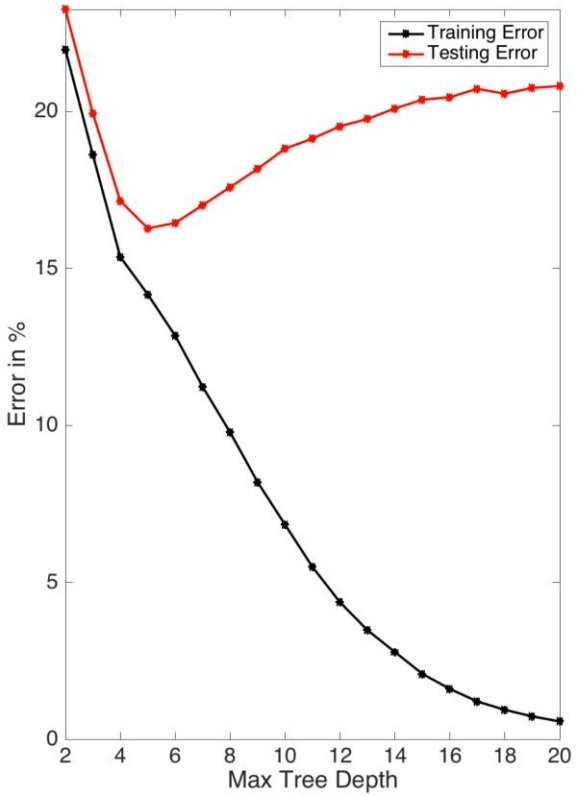
Decision Trees Easily Overfit

- **Solutions: Pruning**
  - **Post-pruning:** Grow a tree to perfect classification on the training set and then prune using a validation set.
    - Prune the tree in a greedy manner
    - Based on a validation set, calculate accuracy using the full tree
    - Pretend to remove a node with all its children from the tree and again measure performance on the pruned tree
      - Can be bottom-up or top-down
    - Remove nodes that lead to highest improvement
    - Repeat until:
      - Further pruning degrades performance
      - **Minimum error:** Prune back to the point where the cross-validated error is a minimum.
      - **Smallest tree.** Prune back slightly further than the minimum error. This pruning creates a decision tree with cross-validation error within 1 standard error of the minimum error. The smaller tree is more intelligible at the cost of a small increase in error.



Decision Trees Easily Overfit

- **Solutions:**
  - Reduce **variance** using **Bagging** (Random Forests)



LUMS

CENTER FOR SPEECH AND LANGUAGE TECHNOLOGIES

Entropy

Agha Ali Raza

CS535/EE514 – Machine Learning

Sources

- **Decision Tree Learning,** Victor Levrenko, Professor at the University of Edinburgh, https://youtube.com/playlist?list=PLBv09BD7ez_4_UoYeGrzvqveIR_USBEKD
- **Machine Learning for Intelligent Systems**, Kilian Weinberger, Cornell, Lecture 16 (videos 28, 29), https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote17.html
- **StatQuest with Josh Starmer**
  - Regression Trees, Clearly Explained!!!, https://www.youtube.com/watch?v=g9c66TUylZ4
  - Entropy (for data science) Clearly Explained!!!, https://www.youtube.com/watch?v=YtebGVx-Fxw
  - Expected Values, Main Ideas!!!, https://www.youtube.com/watch?v=KLs_7b7SKi4
  - StatQuest: Decision Trees, https://www.youtube.com/watch?v=7VeUPuFGJHk
- **Information entropy | Journey into information theory | Computer Science | Khan Academy**, Khan Academy Labs, https://www.youtube.com/watch?v=2s3aJfRr9gE

Quick Revision: Random Variables

- **Random Variables**

A random variable, usually written X, is a variable whose possible values are numerical outcomes of a random phenomenon. There are two types of random variables, **discrete** and **continuous**.

- **Discrete Random Variables**

A discrete random variable takes on only a countable number of distinct values such as 0,1,2,3,4,... Examples of discrete random variables include the number of children in a family, the Friday night attendance at a cinema, the number of patients in a doctor's surgery, the number of defective light bulbs in a box of ten.

The probability distribution of a discrete random variable is a list of probabilities associated with each of its possible values. It is also sometimes called the *probability function* or the *probability mass function*.

Suppose a random variable X may take k different values, with the probability that $X = x_i$ defined to be $P(X = x_i) = p_i$. The probabilities p_i must satisfy the following:

$$0 < p_i < 1 \; for \; each \; i$$
$$p_1 + p_2 + \ldots + p_k = 1$$

http://www.stat.yale.edu/Courses/1997-98/101/ranvar.htm

Quick Revision: Random Variables

- **Continuous Random Variables**

A continuous random variable takes an infinite number of possible values. Examples include height, weight, the amount of sugar in an orange, the time required to run a mile.

A continuous random variable is not defined at specific values. Instead, it is defined over an interval of values, and is represented by the area under a curve (the integral). The probability of observing any single value is equal to 0, since the number of values which may be assumed by the random variable is infinite.

Suppose a random variable X may take all values over an interval of real numbers. Then the probability that X is in the set of outcomes A, $P(A)$, is defined to be the area above A and under a curve. The curve, which represents a function $p(x)$, must satisfy the following:

$$The\ curve\ has\ no\ negative\ values\ (p(x)\ >\ 0\ for\ all\ x)$$
$$The\ total\ area\ under\ the\ curve\ is\ equal\ to\ 1.$$

A curve meeting these requirements is known as a *probability density curve*.

http://www.stat.yale.edu/Courses/1997-98/101/ranvar.htm

Quick Revision: Expected Values

- If X is a ***discrete random variable*** with a finite number of outcomes $\{x_1, x_2, \ldots, x_k\}$ occurring with probabilities $\{p_1, p_2, \ldots, p_k\}$ respectively. The expectation of X is defined as:

$$E[X] = \sum_{i=1}^{k} x_i \, p_i = x_1 p_1 + x_2 p_2 + \cdots + x_k p_k$$

As $p_1 + p_2 + \cdots + p_k = 1$, the expected value is the weighted sum of the x_i values, with the probabilities p_i as the weights.

- If X is a ***continuous random variable*** with a probability density function $f(x)$, then the expected value is defined as:

$$E[X] = \int_{\mathbb{R}} x f(x) dx$$

where the values on both sides are well defined or not well defined simultaneously.

https://en.wikipedia.org/wiki/Expected_value

Entropy

- The weighted average surprise of a random variable or
  - the expected surprise of a random variable or
  - the average number of yes/no questions required to find out the outcome of a random variable unambiguouslyor
  - the average number of bits required to encode all possible outcomes of a random variable
- Assume that there is an event, A, with K possible outcomes ($k = 1 \dots K$)

1. The surprise associated with an outcome $k, Surprise_k$
2. The probability of an outcome k, $P(A = k) = p_k$

$$Entropy(A) = H(A) = \sum_K p_k \, Surprise_k$$

In information theory, the entropy of a random variable is the average level of "information", "surprise", or "uncertainty" inherent in the variable's possible outcomes.

Let's talk about surprise!

You in every ML quiz!

- Let’s try to quantify surprise.

Events in isolation

- Which one of these would be more surprising?
  - The teacher taught about decision trees today
  - The teacher brought an African elephant to the class today
- So, low probability events are more surprising

Probability distributions

- Which one of these are you more uncertain about?
  - Whether the next flip of a fair coin would be a head?
  - A bag contains 9 red and 1 white ball. The next random draw would bring out a red ball.
- You must leave the game early. When will you leave with more suspense?
  - When both teams are equally likely to win
  - When one of the teams is clearly winning
- So, a system where all possible outcomes are equally likely makes us more uncertain **on average** compared to a system with a clearly skewed probability distribution

Information

- Quantifying *information* is the foundation of the field of *information theory*.
- The intuition behind quantifying information is the idea of measuring how much surprise there is in an event. Those events that are rare (low probability) are more surprising and therefore have more information than those events that are common (high probability).
- **Low** Probability Event: **High** Information (surprising).
- **High** Probability Event: **Low** Information (unsurprising).

"The basic intuition behind information theory is that learning that an unlikely event has occurred is more informative than learning that a likely event has occurred."— Page 73, Deep Learning, 2016.

- **Rare events** are **more uncertain** or **more surprising** and require **more information** to represent them than common events.

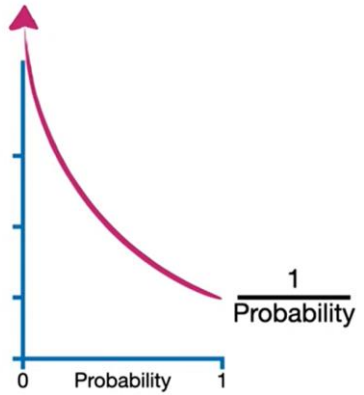
- Therefore, the amount of **information or surprise** associated with an event seems to be **inversely proportional to its probability**

$$Information(x) \propto \frac{1}{p(x)}$$

https://machinelearningmastery.com/what-is-information-entropy/

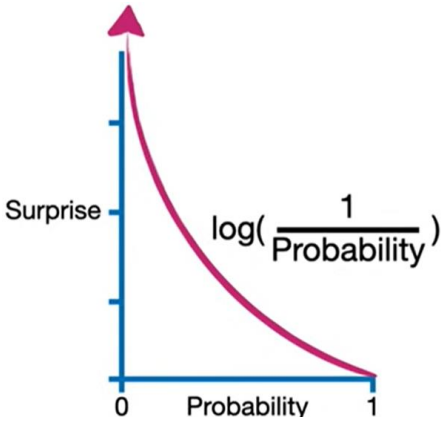
Back to surprise

$$Information(x) \propto \frac{1}{p(x)}$$



Note: This is problematic in this raw form as it varies between ∞ (for $p(x) = 0$) and 1 (for $p(x) = 1$)

- Suppose we have a biased coin that (almost) always gives heads
  - $P(X = tails)$ is very surprising so the surprise should be very high
  - $P(X = heads)$ is not surprising so the surprise should be close to 0
- If we define $surprise(x) = \frac{1}{p(x)}$
  - $Surprise(X = tails) = \frac{1}{0} = \infty$, which is fine
  - $Surprise(X = heads) = \frac{1}{1} = 1$, may not be fine as we want to say "no surprise"
- But if we scale it differently and use $surprise(x) = \log(\frac{1}{p(x)})$
  - $Surprise(X = tails) = \log(\frac{1}{0}) = undefined$, which is fine as it was pretty high approaching 0.
  - $Surprise(X = heads) = \log(\frac{1}{1}) = 0$



Entropy (for data science) Clearly Explained!!!, https://www.youtube.com/watch?v=YtebGVx-Fxw

Summing it up

- Suppose we have a biased coin
- For 100 coin flips, we expect
  - 0.9*100 heads
  - 0.1*100 tails
  - Total surprise = $0.9 * 100 * 0.15 + 0.1 * 100 * 3.32$
  - Average surprise = $\frac{0.9*100*0.15 + 0.1*100*3.32}{100} = 0.9 * 0.15 + 0.1 * 3.32 = 0.467$
  - $E(Surprise) = H(X) = \sum P(X = x) * \log_2(\frac{1}{P(X=x)})$

	Heads	Tails
$P(X)$	0.9	0.1
$\frac{1}{\log_2(P(X))}$	0.15	3.32

- Now, suppose we have an unbiased coin
  - $H(X) = (0.5 * 1) + (0.5 * 1) = 1$

	Heads	Tails
$P(X)$	0.5	0.5
$\frac{1}{\log_2(P(X))}$	1	1

Information

- We can calculate the amount of information there is in an event using the probability of the event.
- This is called “Shannon information,” “self-information,” or simply the “information,” and can be calculated for a discrete event x as follows:

$$Information(x) = \log(\frac{1}{p(x)}) = -\log(\ p(x)\ )$$

- Where $\log()$ is the base-2 logarithm and $p(x)$ is the probability of the event x.

- Base-2 logarithm means that the units of the information measure is in $bits$ (binary digits). If we use base-e, then the units of information are called $nats$.

https://machinelearningmastery.com/what-is-information-entropy/

Uncertainty and information

- A machine is generating four symbols A, B, C and D randomly with equal probabilities, $p_k = \frac{1}{4} = 0.25, \forall k$

Machine 1	A	B	A	D	C	B	C	…

- Another machine is generating four symbols A, B, C and D with probabilities, $p_A = 0.5, p_B = 0.125, p_C = 0.125, p_D = 0.25$

Machine 2	A	A	A	D	C	A	D	…

- Which machine is generating more information?
- When a symbol is released, which machine decreases our uncertainty the most?
- Which machine is more surprising on average?

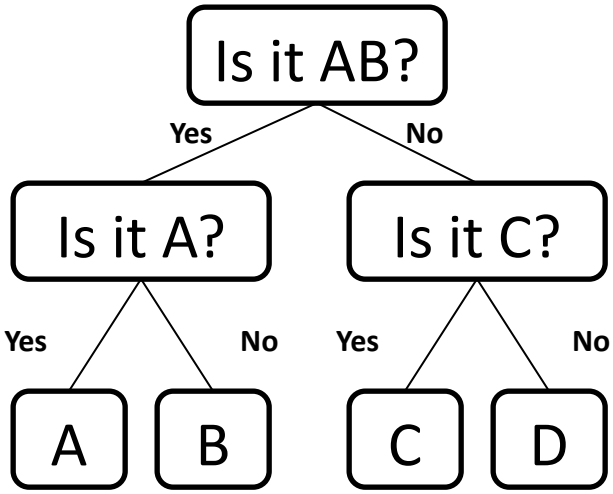
- Let's try to quantify surprise by the average number of yes/no questions that can lead us to the outcome!

Khan Academy: https://www.youtube.com/watch?v=2s3aJfRr9gE

Entropy

Rephrasing: If you have to predict the next symbol from each machine; what is the minimum number of Yes/No questions you would expect to ask?

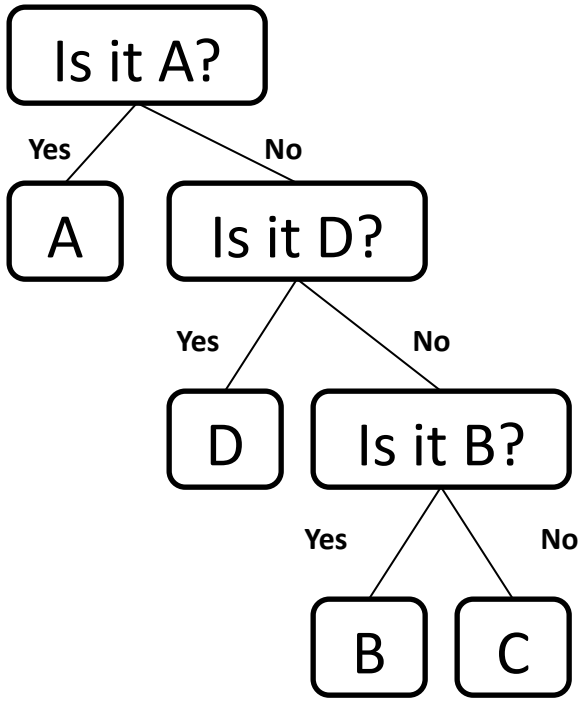
Machine 1



2 questions per symbol or 2 bits

- **To predict 100 symbols, Machine 1 requires 200 questions, while Machine 2 requires 175.**
- **Machine 1 produces more information on average as it reduces more average uncertainty and surprise!**

Machine 2



0.5*1 + 0.25*2 + 0.125*3 + 0.125*3 = 1.75 questions per symbol, or 1.75 bits

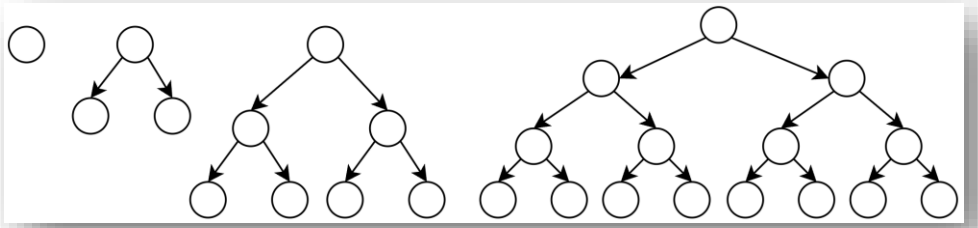
Khan Academy: https://www.youtube.com/watch?v=2s3aJfRr9gE

Outcomes and surprise

$$height\ of\ tree = \log_2(\#leaf\ nodes)$$
$$\#questions = \log_2(\#outcomes)$$
$$\#bits = \log_2 K$$

When all outcomes k among K possible outcomes are equally probable:

$$p_k = \frac{1}{K} \Rightarrow K = \frac{1}{p_k}$$



Therefore

$$\#bits_k = \log_2(\frac{1}{p_k})$$

Now, the total expected surprise across all outcomes, weighted by the probability of each outcome, is:

$$H = \sum_{k=1}^{K} p_k \log_2(\frac{1}{p_k})$$

$$H = -\sum_{k=1}^{K} p_k \log_2(p_k)$$

So, entropy is the total surprise across all outcomes, weighted by the probability of each outcome.

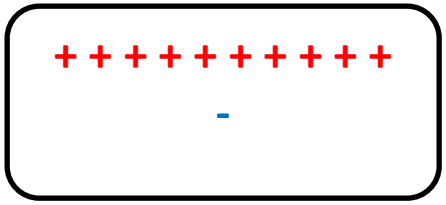
Entropy

- Entropy is the highest for a system where all outcomes are equally likely
  - Your team winning a match that is stuck till the last minute
- Entropy is low when some outcomes are more likely
  - Your team winning a match when the opposition played poorly in the first innings
- Predictability reduces the entropy / surprise / information
- An attribute has high mutual information with an event, if revealing it would decrease our surprise / entropy / uncertainty about the outcomes of A, significantly
- If the entropy of the information source drops, we can ask fewer questions to guess the outcome
- The bit here is our measure of information OR measure of surprise
- When we use natural logs ($\log_e$) the unit is *nats*.

Entropy

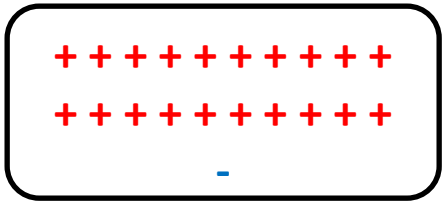
$$H(S) = \sum_{k=1}^{c} p_k \log_2\left(\frac{1}{p_k}\right)$$

$$p_+ = \frac{10}{11} = 0.91, \qquad p_- = \frac{1}{11} = 0.09,$$



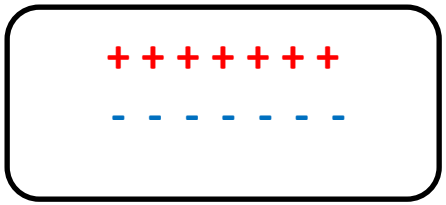
$$H = 0.91 * \log_2\left(\frac{1}{0.91}\right) + 0.09 * \log_2\left(\frac{1}{0.09}\right) = 0.91 * \mathbf{0.14} + 0.09 * \mathbf{3.45} = 0.43$$

$$p_+ = \frac{20}{21} = 0.95, \qquad p_- = \frac{1}{21} = 0.047,$$



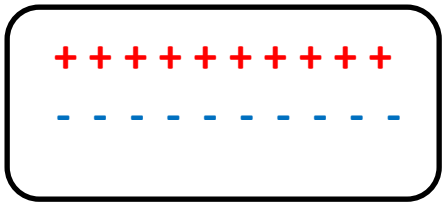
$$H = 0.95 * \log_2\left(\frac{1}{0.95}\right) + 0.047 * \log_2\left(\frac{1}{0.047}\right) = 0.95 * \mathbf{0.07} + 0.047 * \mathbf{4.39} = 0.28$$

$$p_+ = \frac{7}{14} = 0.5, \qquad p_- = \frac{7}{14} = 0.5,$$



$$H = 0.5 * \log_2\left(\frac{1}{0.5}\right) + 0.5 * \log_2\left(\frac{1}{0.5}\right) = 0.5 * \mathbf{1} + 0.5 * \mathbf{1} = 1.0$$

$$p_+ = \frac{10}{20} = 0.5, \qquad p_- = \frac{10}{20} = 0.5,$$



$$H = 0.5 * \log_2\left(\frac{1}{0.5}\right) + 0.5 * \log_2\left(\frac{1}{0.5}\right) = 0.5 * \mathbf{1} + 0.5 * \mathbf{1} = 1.0$$

Evaluating Language Models

- Models that assign probabilities to sequences of words are called **language models** or **LMs**.
- If we are given a corpus of text and want to compare two different language models:
  - We divide the data into training and test sets
  - Train the parameters of both models on the training set, and
  - Compare how well the two trained models **fit the test set**.
- Goodness of fit is simply
  - Whichever model assigns a higher probability to the test set
  - Meaning it more accurately predicts the test set
  - Meaning it is less surprised by the test set

https://web.stanford.edu/~jurafsky/slp3/3.pdf

Comparing Probability Distributions using Cross Entropy

The cross entropy is the average number of bits needed to encode data coming from a source with distribution p when we use model q – *Page 57, Machine Learning: A Probabilistic Perspective, 2012.*

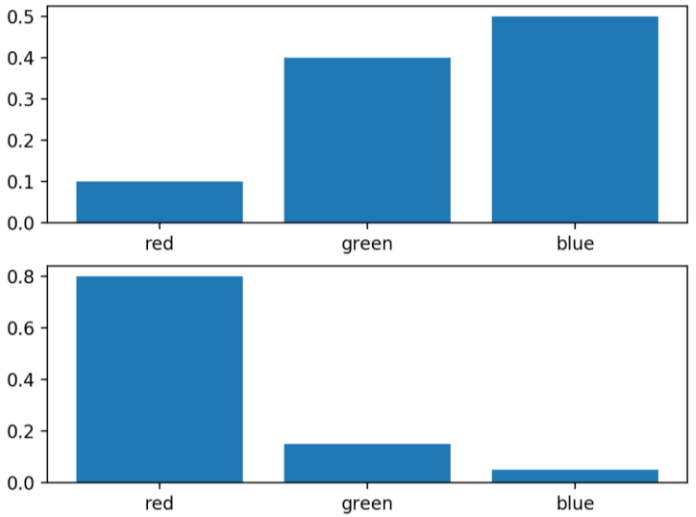
$$Entropy(p, q) = H(p, q) = \sum_{K} p_k \, Surprise(q_k)$$

$$H(p, q) = \sum_{k=1}^{K} p_k \log_2(\frac{1}{q_k})$$

$$H(p, q) = -\sum_{k=1}^{K} p_k \log_2(q_k)$$

That is, we draw sequences according to the probability distribution p, but sum the log of their probabilities according to q.

Cross Entropy



events = ['red', 'green', 'blue']

p = [0.10, 0.40, 0.50]

q = [0.80, 0.15, 0.05]

$$H(p,q)\colon 3.288\ bits$$
$$H(q,p)\colon 2.906\ bits$$

$$H(p,p)\colon 1.361\ bits$$

Essentially:

$$H(p,p) = \sum_{k=1}^{K} p_k \log_2\left(\frac{1}{p_k}\right) = H(p)$$

$$H(q,q) = H(q)\colon 0.884\ bits$$

$$H(p,p) \leq H(p,q)$$

i.e.

$$H(p) \leq H(p,q)$$

https://machinelearningmastery.com/cross-entropy-for-machine-learning/

Cross Entropy

- The more accurately q models p, the closer the cross-entropy $H(p,q)$ will be to the true entropy $H(p)$
- Thus, the difference between $H(p,q)$ and $H(p)$ is a measure of how accurate a model is.
- Between two models q_1 and q_2 of p, the more accurate model will be the one with the lower cross-entropy with p
- $Perplexity(W) = 2^{H(W)}$

https://web.stanford.edu/~jurafsky/slp3/3.pdf

LUMS

CENTER FOR SPEECH AND LANGUAGE TECHNOLOGIES

Huffman Coding

Agha Ali Raza

CS535/EE514 – Machine Learning

Huffman Coding

Huffman Coding (Algorthms, 3rd Ed, Cormen)

Huffman coding is an **optimal, variable-length, prefix coding** scheme that is commonly used for **lossless** data compression.

	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101
Variable-length codeword	0	101	100	111	1101	1100

Figure 16.3 A character-coding problem. A data file of 100,000 characters contains only the characters `a`–`f`, with the frequencies indicated. If we assign each character a 3-bit codeword, we can encode the file in 300,000 bits. Using the variable-length code shown, we can encode the file in only 224,000 bits.

Huffman Coding (Algorthms, 3rd Ed, Cormen)

	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101
Variable-length codeword	0	101	100	111	1101	1100

If we use a ***fixed-length code***, we need 3 bits to represent 6 characters: `a` = 000, `b` = 001, . . . , `f` = 101. This method requires 300,000 bits to code the entire file. Can we do better?

A ***variable-length code*** can do considerably better than a fixed-length code, by giving frequent characters short codewords and infrequent characters long codewords. Figure 16.3 shows such a code; here the 1-bit string 0 represents `a`, and the 4-bit string 1100 represents `f`. This code requires

$$(45 \cdot 1 + 13 \cdot 3 + 12 \cdot 3 + 16 \cdot 3 + 9 \cdot 4 + 5 \cdot 4) \cdot 1{,}000 = 224{,}000 \text{ bits}$$

to represent the file, a savings of approximately 25%. In fact, this is an optimal character code for this file, as we shall see.

Huffman Coding (Algorithms, 3rd Ed, Cormen)

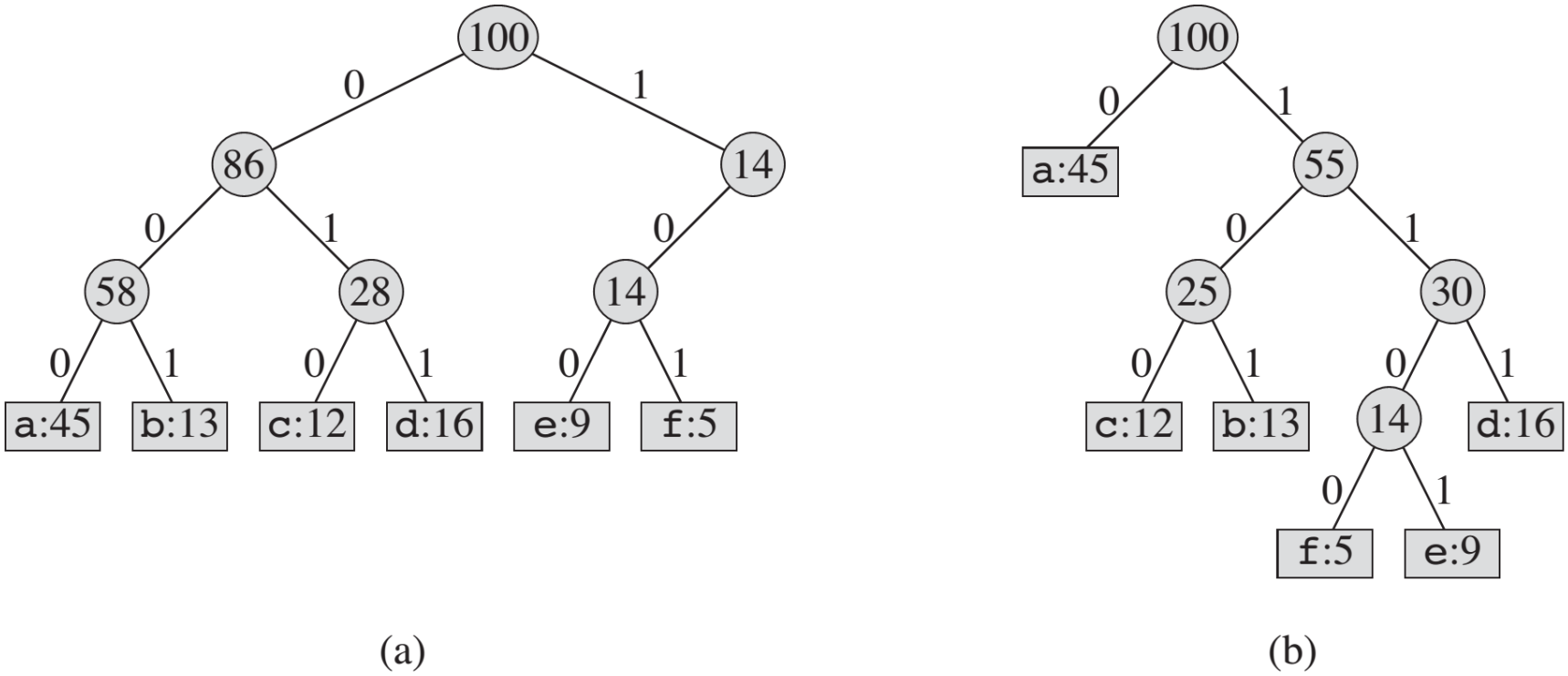
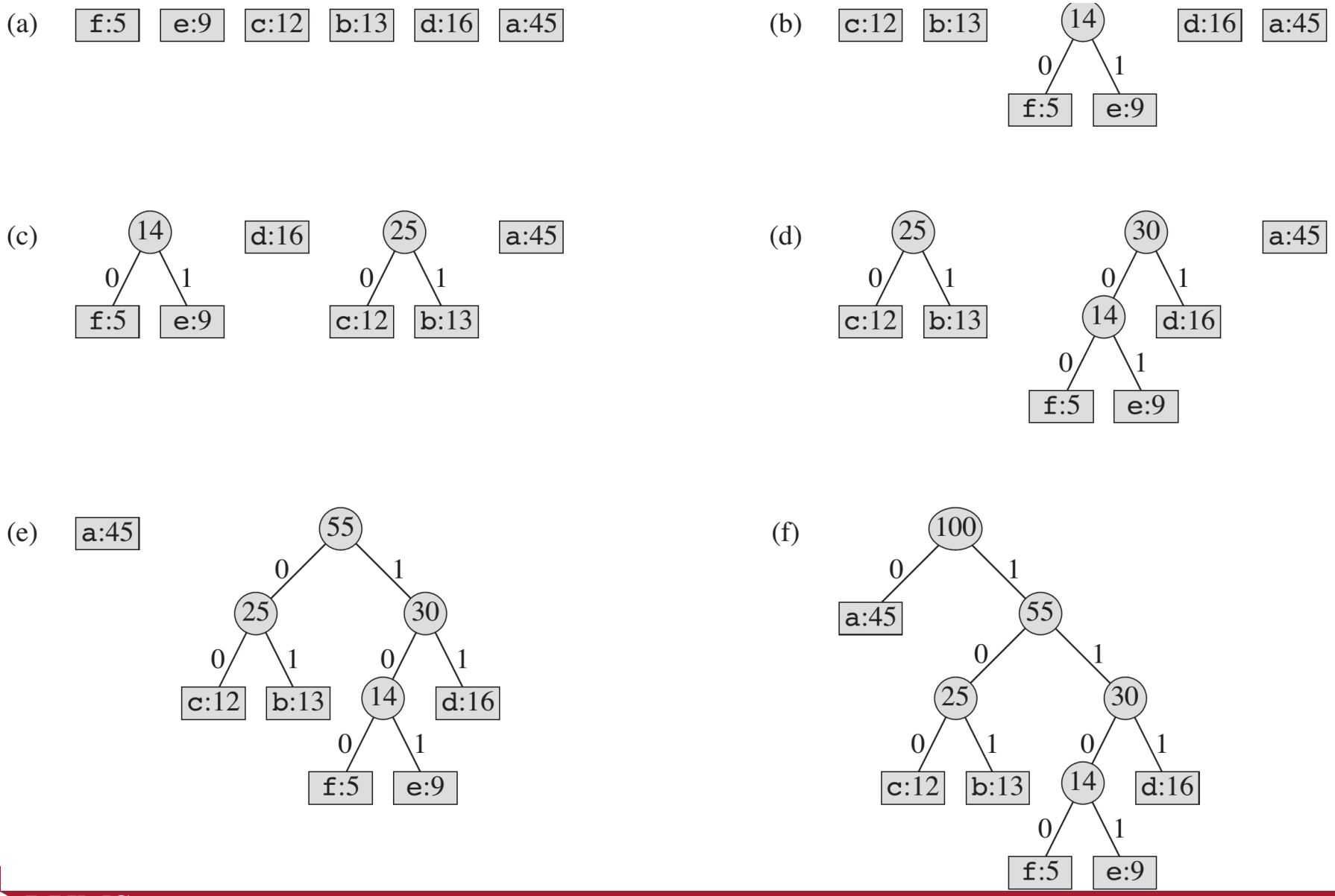


Figure 16.4 Trees corresponding to the coding schemes in Figure 16.3. Each leaf is labeled with a character and its frequency of occurrence. Each internal node is labeled with the sum of the frequencies of the leaves in its subtree. **(a)** The tree corresponding to the fixed-length code a = 000, . . . , f = 101. **(b)** The tree corresponding to the optimal prefix code a = 0, b = 101, . . . , f = 1100.

Huffman Coding (Algorithms, 3rd Ed, Cormen)



LUMS

CENTER FOR SPEECH AND LANGUAGE TECHNOLOGIES

Parametric vs Non-parametric

Agha Ali Raza

CS535/EE514 – Machine Learning

A discussion on Parametric/Non-parametric models

Families of Algorithms

- Parametric vs. non-parametric.

Parametric Algorithms: A parametric algorithm is one that has a **constant set of parameters, which is independent of the number of training samples**.

- The amount of space needed to store the trained classifier.
- E.g., the perceptron algorithm, or logistic regression with w and b as the parameters, which define the separating hyperplane.
  - The dimension of w depends of the dimension of the training data, but not on how many training samples you use for training.

Non-parametric Algorithms: The number of parameters of a non-parametric algorithm **scales as a function of the training samples**.

- E.g., the k-Nearest Neighbors classifier where during "training" we store the entire training data
  - so the parameters that we learn are identical to the training set and the number of parameters (i.e., the storage we require) grows linearly with the training set size.

https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote17.html

A discussion on Parametric/Non-parametric models

- Decision Trees are an interesting case
  - If they are trained to full depth, they are non-parametric, as the depth of a decision tree scales as a function of the training data (in practice $O(\log_2 n)$.
  - If we however limit the tree depth by a maximum value, they become parametric (as an upper bound of the model size is now known prior to observing the training data).

https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote17.html

For more details please visit

http://aghaaliraza.com

Thank you!